

TITLE
DESIGN AND IMPLEMENTATION OF UNIVERSITY SECURITY
MANAGEMENT SYSTEM

BY

GIDEON THLAMA

IFS/23U/3492

JUNE, 2026

A Project Submitted to the Department of Information System, Faculty of Computing,
Nigerian Army University, Biu, in Partial Fulfilment of the Requirements for the
Award of Bachelor of Science B.Sc. Information System

DECLARATION

I hereby declare that this project titled “University Security Management System.” is an original work carried out by me under the supervision of my project supervisor. This work has not been submitted to any institution for the award of any degree or certificate.

Name: GIDEON THLAMA

Signature: _____

Matric Number: IFS/23U/3492

Date: _____

CERTIFICATION

This is to certify that University Security Management System was carried out by Gideon Thlama and has been approved for submission to the Department of Information System, Faculty of Computing, Nigerian Army University, Biu.

MRS. MARYAM WAZIRI MOHAMMED

Supervisor

Signature/Date

MR. DAMU LIMAN MOHAMMED

Project Coordinator

Signature/Date

DR. MIKAILU HABILA

Head of Department

Signature/Date

External Examiner

Signature/Date

DEDICATION

This project is dedicated to Almighty God for His grace, wisdom, protection, and strength throughout the course of this academic journey. It is also dedicated to my parents, family members, lecturers, friends, and every individual who supported and encouraged me throughout the development of this research work.

ACKNOWLEDGEMENT

I sincerely appreciate Almighty God for granting me life, strength, wisdom, and understanding throughout the duration of this project work. My profound gratitude goes to my project supervisor MRS. MARYAM WAZIRI MOHAMMED for the guidance, encouragement, corrections, and support provided during the course of this research. I also appreciate the Head of Department DR. MIKAILU HABILA, lecturers, and staff members of the Department of Information System, Nigerian Army University, Biu, for the knowledge and training imparted in me. Special appreciation goes to my parents and family for their prayers, financial support, encouragement, and sacrifices throughout my studies. I also thank my colleagues and friends who contributed positively toward the successful completion of this project.

TABLE OF CONTENT

TITLE	i
DECLARATION.....	ii
CERTIFICATION	iii
DEDICATION	iv
ACKNOWLEDGEMENT.....	v
TABLE OF CONTENT	vi
ABSTRACT	xi
1.1 Background of the Study	1
1.2 Statement of the Problem	3
1.3 Aim and Objectives of the Study	4
<i>1.3.1 Aim</i>	4
<i>1.3.2 Objectives</i>	4
1.4 Scope of the Study	5
Excluded from Scope.....	6
1.5 Significance of the Study	6
1.6 Definition of Terms	7
CHAPTER TWO LITERATURE REVIEW	9
2.1 Introduction	9
2.2 Conceptual Framework	9
2.3 Artificial Intelligence	10
2.4 Computer Vision	10
2.5 Facial Recognition Technology	11
2.6 Smart Surveillance Systems	12
2.7 Access Control Systems	12
2.8 Deep Learning and Classical Approaches to Facial Recognition	12
2.9 Review of Existing Systems	14
Study by Oladele et al. (2024).....	14
Study by Srikanth et al. (2024).....	14
Study by Krishna and Manoj (2024)	14
Study by Ravi Sankar (2025).....	14
Study by O'Leary (2025).....	15

Study by Pham-Quoc, Loc and An (2024).....	15
2.10 Comparative Analysis of Existing Systems	15
2.11 Theoretical Framework	15
Biometric Recognition Theory	16
Technology Acceptance Model (TAM)	16
Situational Crime Prevention Theory	16
2.12 Research Gaps	16
2.13 Summary of Literature Review.....	17
CHAPTER THREE SYSTEM ANALYSIS AND DESIGN	19
3.1 Introduction	19
3.2 Methodology Adopted	19
Reasons for Choosing Agile Methodology.....	19
Agile Development Phases and Schedule	20
3.3 Analysis of Existing System	20
Description of Existing System.....	20
Problems of Existing Systems	20
3.4 Requirements Analysis.....	21
Functional Requirements.....	21
Non-Functional Requirements	22
3.5 Proposed System Design	22
3.6 System Architecture	23
Layer 1: Hardware Layer	23
Layer 2: Processing Layer.....	24
Layer 3: Application Layer.....	24
Layer 4: Presentation Layer	24
3.7 Data Flow Diagram (DFD)	24
3.8 Use Case Diagram.....	27
Actors	27
Administrator Use Cases	28
Security Personnel Use Cases	28
Student and Staff Use Cases	28
3.9 Entity Relationship Diagram (ERD).....	28

Entities and Attributes	29
Relationships	30
3.10 System Flowchart	30
3.11 Database Design	31
<i>Users Table Structure</i>	31
<i>Access Log Table Structure</i>	32
<i>Security Alert Table Structure</i>	32
<i>Unknown Person Table Structure</i>	32
<i>Camera Table Structure</i>	32
System User Table Structure	33
Incident Report Table Structure	33
3.12 Advantages of the Proposed System	33
3.13 Summary	34
CHAPTER FOUR	35
4.1 System Overview	35
4.2 Software Development Environment	35
4.3 System Modules	37
4.3.1 Authentication and Access Control Module	37
4.3.2 User Management Module	37
4.3.3 Facial Recognition Module	38
4.3.4 Live Surveillance and Monitoring Module	38
4.3.5 Access Control and Logging Module	38
4.3.6 Alert and Notification Module	39
4.3.7 Camera Management Module	39
4.3.8 Incident Management Module	39
4.3.9 Reporting and Analytics Module	39
4.4 User Interface Design	40
4.4.1 Login Page	40
4.4.2 Dashboard (Security Operations Center)	40
4.4.3 Register User Page	41
4.4.4 Face Registration / Photo Gallery Pages	42
4.4.5 Live Monitoring Page	42

4.4.6 Alerts Page	43
4.4.7 Access Logs, Incidents, and Reports Pages	44
4.4.8 Camera and User Management Pages	45
4.5 System Implementation.....	46
4.5.1 Backend Implementation	46
4.5.2 Frontend Implementation	47
4.5.3 Database and Storage Implementation.....	47
4.5.4 API Integration.....	48
4.5.5 Authentication System	48
4.5.6 AI Model Integration	49
4.6 System Testing	50
4.6.1 Unit Testing.....	50
4.6.2 Integration Testing	51
4.6.3 System Testing	51
4.6.4 User Acceptance Testing	52
4.7 Performance Evaluation	53
4.8 Discussion of Results	54
4.9 Chapter Summary	56
CHAPTER FIVE	57
5.1 Summary	57
5.2 Conclusion	58
5.3 Achievements of Objectives	58
5.4 Contributions to Knowledge.....	59
5.5 Limitations of the Study.....	59
5.6 Recommendations.....	60
5.7 Suggestions for Further Research.....	60
REFERENCES	62
APPENDICES.....	64
APPENDIX A: Source Code Snippets.....	64
A.1 Face Detection (ai_engine/face_detector.py)	64
A.2 Recognition Confidence Threshold (ai_engine/face_recognition_engine.py).....	64
A.3 Role-Based Access Control Decorators (utils/decorators.py).....	65

A.4 Unknown-Person Alert Creation (services/alert_service.py)	65
APPENDIX B: Test Cases and Results	66
APPENDIX C: User Manual	67
C.1 Installation	67
C.2 Setup	67
C.3 Usage	67
C.4 Troubleshooting	68
APPENDIX D: Screenshots	68
APPENDIX E: System Requirements	70
E.1 Hardware Requirements	70
E.2 Software Requirements	70
APPENDIX F: Sample Data	70

ABSTRACT

This project presents the design and implementation of University Security Management System. The study was motivated by the increasing need for intelligent campus security, real-time surveillance, and automated identity verification within university environments. Traditional security systems rely heavily on security personnel and conventional CCTV cameras that only record events without providing intelligent analysis or real-time threat detection. The proposed system utilizes Artificial Intelligence, Computer Vision, and Facial Recognition technologies to monitor university environments, identify authorized individuals, detect unknown persons, and generate real-time security alerts. The system was developed using Python, Flask Framework, OpenCV, DeepFace, HTML, CSS, JavaScript, and PostgreSQL Database. The system continuously captures video streams from surveillance cameras, detects faces, extracts facial features, and compares them with records stored in the database. Authorized students and staff are automatically identified and logged, while unknown individuals trigger immediate security notifications. The system also maintains comprehensive access records and surveillance logs for administrative and security purposes. The implementation results demonstrated effective real-time monitoring, improved access control, enhanced campus security, and efficient identity verification. The system successfully reduced dependence on manual monitoring and provided a centralized platform for intelligent security management. The study concludes that university security management system can significantly strengthen campus security and improve operational efficiency within universities. It is recommended that educational institutions adopt intelligent surveillance systems to improve safety, prevent unauthorized access, and support proactive security management. Keywords: Artificial Intelligence, Facial Recognition, Smart Surveillance, Access Control, Security Monitoring, Computer Vision, Campus Security.

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

The rapid advancement of Artificial Intelligence (AI), Computer Vision, and Biometric Authentication technologies has transformed the way organizations manage security, surveillance, and access control. Educational institutions, particularly universities, are increasingly adopting intelligent technologies to enhance campus security, monitor activities, and protect students, staff, and infrastructure from security threats.

Traditional security systems commonly deployed in universities rely heavily on human security personnel, identity cards, visitor registers, and conventional Closed-Circuit Television (CCTV) cameras. While these methods provide a basic level of protection, they suffer from several limitations including human error, delayed incident response, unauthorized access, and the inability to perform intelligent identification of individuals. Conventional CCTV systems primarily record events for later review and lack the capability to automatically identify authorized persons or detect potential security threats in real time.

Recent developments in Artificial Intelligence have enabled the creation of smart surveillance systems capable of analyzing video feeds, recognizing faces, detecting suspicious activities, and generating automated security alerts. Facial recognition technology, a branch of biometric authentication, has emerged as one of the most effective methods for identity verification because it uses unique facial characteristics to distinguish individuals. Unlike traditional authentication methods such as identity cards and passwords, facial recognition provides a contactless, secure, and efficient means of verifying identity.

According to Oladele et al. (2024), AI-based facial recognition systems provide accurate and reliable identification mechanisms capable of reducing impersonation and improving operational efficiency. Similarly, Srikanth et al. (2024) observed that modern multi-face recognition systems can identify multiple individuals simultaneously in real time, making them suitable for large-scale environments such as universities.

The emergence of smart surveillance technologies has further enhanced the capabilities of facial recognition systems. Ravi Sankar (2025) emphasized that AI-driven security systems integrated with biometric authentication and intelligent access control mechanisms significantly improve security management by automating monitoring and alert generation. Likewise, O'Leary (2025) highlighted that AI-powered identity and access management solutions improve authentication accuracy and strengthen organizational security through automated identity verification processes.

In educational environments, access control and surveillance have become increasingly important due to rising concerns regarding unauthorized access, theft, vandalism, impersonation, and other security-related incidents. Universities often accommodate thousands of students, staff, and visitors daily, making manual monitoring both difficult and inefficient. As campus populations continue to grow, there is an increasing need for intelligent systems capable of automatically verifying identities, monitoring activities, and responding to security events in real time.

Pham-Quoc, Loc, and An (2024) proposed an Edge AI-powered access control system for smart classrooms and concluded that intelligent access management technologies improve security, reduce administrative workload, and enhance monitoring efficiency. Their findings demonstrate the growing importance of integrating Artificial Intelligence with access control systems within educational institutions.

Despite these technological advancements, many universities in developing countries still depend on traditional security approaches that provide limited automation and intelligence. The absence of integrated surveillance and access control platforms creates vulnerabilities that can compromise campus safety and operational efficiency.

To address these challenges, this project proposes the design and implementation of a prototype AI-powered smart surveillance and access control system for universities, named SKYNET. The proposed system combines Artificial Intelligence, classical Computer Vision-based facial recognition, and a centralized database to create an intelligent security platform capable of identifying authorized individuals, detecting unknown persons, monitoring campus activities, generating security alerts, and maintaining comprehensive access records in real time.

1.2 Statement of the Problem

Security remains one of the most critical challenges facing universities and higher educational institutions. Many institutions continue to rely on conventional security methods such as security guards, manual visitor verification procedures, identity cards, and standard CCTV surveillance systems. Although these methods provide a certain level of protection, they are often insufficient in addressing modern security challenges.

The major problems identified in existing university security systems include:

1. **Unauthorized Access to Campus Facilities:** Many university environments lack effective mechanisms for verifying the identities of individuals entering restricted areas. Unauthorized persons can gain access to classrooms, laboratories, hostels, and administrative buildings without proper verification.
2. **Dependence on Human Monitoring:** Security personnel are responsible for monitoring large environments and multiple surveillance screens simultaneously. Human operators may experience fatigue, distraction, or reduced concentration, leading to missed security incidents.

3. Lack of Intelligent Surveillance: Conventional CCTV systems only record video footage without analyzing it. Security personnel often review recordings after incidents have already occurred, limiting the effectiveness of incident prevention and response.

4. Absence of Automated Identity Verification: Many institutions rely on identification cards and manual checks that can be forged, shared, or manipulated. This creates opportunities for impersonation and unauthorized access.

5. Poor Record Management: Security logs and visitor records are often maintained manually, making them difficult to organize, retrieve, and analyze during investigations.

Therefore, there is a need for an intelligent, automated, and integrated security platform capable of providing real-time surveillance, facial recognition-based identity verification, access control, security alert generation, and centralized security management within university environments.

1.3 Aim and Objectives of the Study

1.3.1 Aim

The aim of this project is to design and implement a prototype of a University Security Management System that demonstrates AI-powered facial recognition for access control.

1.3.2 Objectives

The specific objectives of the study are:

- i. To develop a prototype AI-powered facial recognition system capable of accurately identifying authorized students and staff in real time.
- ii. To design a prototype intelligent surveillance system for continuous monitoring of university environments.

- iii. To implement an automated access control mechanism based on facial recognition technology, to detect unknown and unauthorized individuals and generate immediate security alerts.
- iv. To develop a centralized database for storing user records, access logs, surveillance data, and security alerts.
- v. To provide a web-based Security Operations Center (SOC) dashboard for monitoring and managing security activities and improve campus security by reducing unauthorized access and enhancing incident response capabilities.

1.4 Scope of the Study

This study is scoped to the design and implementation of a prototype facial recognition-based identification, real-time surveillance, automated access logging, unknown-person detection, security alert generation, centralized database management, a web-based security dashboard, report generation, and activity audit trails for a single university campus. As a prototype, the system is developed and tested under the following limitations. The system is tested with a limited number of registered users and a sample/simulated dataset, rather than the full student and staff population of the university.

The system operates with a single USB webcam, automatically identified by its DirectShow device name (e.g., a name containing “USB”) rather than a fixed, hardware-dependent camera index, rather than a full multi-camera, campus-wide deployment of IP or CCTV cameras. While the platform's database and dashboard support registering multiple camera records to model future expansion, the prototype currently captures and displays a single live physical video feed across all registered camera tiles.

Testing and evaluation are conducted in a controlled or lab environment rather than on a live, production university network.

The system demonstrates core functionality and feasibility rather than serving as a fully deployed, production-grade security system ready for institution-wide rollout.

It excludes mobile application development, integration with national identity databases, weapon detection, drone-based surveillance, law enforcement integration, non-facial biometric methods, and hardware manufacturing.

Excluded from Scope

Mobile application development, integration with national identity databases, advanced weapon detection systems, drone-based surveillance systems, law enforcement integration, biometric methods other than facial recognition, hardware manufacturing of surveillance devices, and full-scale, institution-wide production deployment.

1.5 Significance of the Study

This study is significant for several reasons:

1. **Enhanced Campus Security:** The prototype demonstrates the potential to improve campus safety through intelligent surveillance, facial recognition, and automated threat detection.
2. **Improved Access Control:** The system shows how only authorized individuals can be recognized and logged, reducing unauthorized access if deployed at scale.
3. **Real-Time Monitoring:** The prototype demonstrates continuous monitoring of a limited environment and provides instant alerts, including an audible sound alert on the monitoring dashboard, when suspicious activities occur.
4. **Administrative Efficiency:** Automated logging and reporting illustrate how the workload of security personnel and administrators could be reduced in a full deployment.
5. **Technological Advancement:** The study demonstrates the practical application of Artificial Intelligence, classical Computer Vision, and facial recognition technologies in solving real-world security challenges using a lightweight, dependency-light implementation.

6. **Research Contribution:** The project contributes to existing knowledge in intelligent surveillance systems, access control technologies, and AI-based campus security solutions.

7. **Scalability:** The prototype is designed with the potential to be expanded to support multiple physical cameras, buildings, and campus locations, and a production-grade database engine, in future work.

1.6 Definition of Terms

Artificial Intelligence (AI): The simulation of human intelligence by computer systems capable of learning, reasoning, and decision-making.

Facial Recognition: A biometric technology that identifies individuals by analyzing unique facial characteristics.

Computer Vision: A field of Artificial Intelligence that enables computers to interpret and analyze visual information from images and videos.

Smart Surveillance: The use of intelligent technologies to automatically monitor, analyze, and respond to events within an environment.

Access Control: A security mechanism used to regulate and monitor entry to physical or digital resources.

Biometric Authentication: The process of verifying identity using unique biological characteristics such as facial features.

Security Alert: An automated notification, including a visual dashboard entry and an audible sound alert, generated when suspicious or unauthorized activities are detected.

Database Management System (DBMS): Software used to store, organize, retrieve, and manage data efficiently.

Unknown Individual: A person detected by the system whose facial features do not match any authorized record in the database.

Security Operations Center (SOC): A centralized platform used to monitor, manage, and respond to security events and alerts.

CHAPTER TWO LITERATURE REVIEW

2.1 Introduction

This chapter presents a comprehensive review of literature related to Artificial Intelligence, Facial Recognition Technology, Smart Surveillance Systems, Access Control Systems, Computer Vision, Machine Learning, and Campus Security Management. The review examines theoretical concepts, existing technologies, empirical studies, and previous research relevant to the development of the university security management system prototype.

The purpose of this chapter is to establish the theoretical foundation of the study, identify existing research gaps, and justify the need for an intelligent security platform capable of integrating surveillance, facial recognition, identity verification, access control, and automated security alert generation.

2.2 Conceptual Framework

The conceptual framework of this study is built around five major components that work together to provide intelligent security monitoring and access control.

1. Artificial Intelligence Layer: This layer provides intelligent decision-making capabilities through machine learning and classical pattern-recognition algorithms.
2. Facial Recognition Layer: This layer performs face detection, feature extraction, facial encoding, and identity verification.
3. Smart Surveillance Layer: This component continuously monitors video streams and analyzes activities within the university environment.
4. Access Control Layer: This layer determines whether detected individuals are authorized or unauthorized.
5. Security Management Layer: This layer generates alerts, stores logs, manages records, and provides reporting functionalities.

The AI Security Platform integrates these five layers into a unified architecture. The Facial Recognition Module handles face detection, face alignment, feature extraction, and identity verification. The Smart Surveillance Module manages live camera monitoring, activity tracking, person detection, and event logging. The Access Control Module handles authorized user verification, unauthorized person detection, access logging, and security alert generation. The Database Management Module stores user records, access logs, security logs, and alert history. Together, these modules produce outputs including security alerts, access reports, surveillance reports, and audit logs.

2.3 Artificial Intelligence

Artificial Intelligence (AI) refers to the ability of computer systems to perform tasks that normally require human intelligence. These tasks include learning, reasoning, decision-making, pattern recognition, and problem-solving.

According to Russell and Norvig (2021), AI systems can be categorized into Narrow AI, General AI, and Super AI. Most practical applications currently fall under Narrow AI because they are designed to perform specific tasks efficiently. AI has transformed various sectors including healthcare, finance, transportation, education, and security. In security applications, AI enables intelligent surveillance, automated threat detection, identity verification, and predictive analysis.

Modern security systems utilize AI algorithms to analyze video streams, detect suspicious behavior, identify individuals, and generate alerts without constant human supervision. The system utilizes Narrow AI through facial recognition and intelligent surveillance algorithms to improve campus security and access control.

2.4 Computer Vision

Computer Vision is a branch of Artificial Intelligence that enables computers to interpret and understand visual information from images and video streams. Computer

vision systems perform tasks such as object detection, facial detection, facial recognition, motion tracking, image classification, and activity recognition.

According to Kushwaha et al. (2023), computer vision technologies have significantly improved the accuracy and efficiency of modern surveillance systems. The development of powerful computer vision libraries such as OpenCV has enabled real-time processing of visual information for various applications including surveillance, autonomous vehicles, healthcare imaging, and biometric authentication. In the system, OpenCV is utilized for real-time video processing, face detection, and facial recognition.

2.5 Facial Recognition Technology

Facial Recognition Technology is a biometric authentication method used to identify or verify individuals based on their facial characteristics. The technology operates by comparing captured facial images against stored facial templates within a database.

According to Jothi et al. (2023), facial recognition has become one of the most reliable biometric identification methods due to its non-contact nature, speed, and high accuracy. The technology operates through five major stages: Face Detection (identifying the presence of a face within an image or video frame), Face Alignment (adjusting the detected face to a standard orientation), Feature Extraction (extracting unique facial features and converting them into numerical representations), Feature Matching (comparing extracted features with stored records), and Identity Verification (determining whether a match exists).

The advantages of facial recognition technology include contactless authentication, fast identification, reduced impersonation, improved security, and real-time operation. The proposed system employs a Haar cascade classifier for face detection and a Local Binary Pattern Histogram (LBPH) recognizer for facial recognition. This classical, statistically grounded pipeline avoids the large pretrained-weight downloads, GPU acceleration, and network dependency associated with deep-learning pipelines such

as DeepFace or ArcFace, making it well suited to a lightweight, offline-capable university prototype (Viola & Jones, 2004; Ahonen et al., 2006).

2.6 Smart Surveillance Systems

Smart surveillance systems combine Artificial Intelligence, computer vision, and sensor technologies to provide automated monitoring and security management. Unlike conventional CCTV systems that simply record footage, smart surveillance systems analyze video streams and generate intelligent responses.

According to Chen et al. (2023), AI-powered surveillance systems can significantly reduce false alarms while improving threat detection capabilities. Smart surveillance systems are capable of monitoring activities continuously, detecting suspicious individuals, recognizing authorized personnel, generating alerts automatically, and maintaining digital audit trails. The platform utilizes smart surveillance techniques to monitor university premises and improve incident response capabilities.

2.7 Access Control Systems

Access Control Systems are security mechanisms designed to regulate entry into protected areas. Traditional access control methods include keys, identity cards, PIN codes, and passwords. Modern systems increasingly utilize biometric authentication methods due to their superior security and convenience.

According to O'Leary (2025), AI-powered identity and access management systems significantly improve authentication accuracy and strengthen organizational security. Access control systems typically perform the following functions: identity verification, authentication, authorization, access logging, and security monitoring. The system implements facial recognition-based access control to verify authorized individuals and detect unauthorized persons.

2.8 Deep Learning and Classical Approaches to Facial Recognition

Deep Learning is a subset of Machine Learning that uses neural networks with multiple layers to learn complex patterns from data. Deep learning has dramatically improved facial recognition accuracy over traditional methods on large, unconstrained benchmark datasets. Several prominent deep-learning models have been developed, including FaceNet by Google (99.63% accuracy), VGG-Face by Oxford University (98.95% accuracy), DeepFace by Facebook (97.35% accuracy), ArcFace by Imperial College London (99.83% accuracy), and OpenFace by Carnegie Mellon University (93.80% accuracy). According to Deng et al. (2019), ArcFace introduces additive angular margin loss that significantly improves facial recognition performance.

Model	Developer	Accuracy
FaceNet	Google	99.63%
VGG-Face	Oxford University	98.95%
DeepFace	Facebook	97.35%
ArcFace	Imperial College	99.83%
OpenFace	Carnegie Mellon	93.80%

Comparison of Deep Learning Facial Recognition Models (Deng et al., 2019)

While the deep-learning models above report the highest published accuracies, they typically require large pretrained-weight files (often several hundred megabytes), GPU acceleration to run at real-time frame rates, and, in many implementations, a network connection to fetch model weights on first use — constraints that are difficult to guarantee in a low-resource prototype or an offline campus deployment.

Classical, statistically grounded methods provide a lighter-weight alternative that predates the deep-learning era but remains widely used where compute and connectivity are constrained. Viola and Jones (2004) introduced a cascade of boosted Haar-like feature classifiers that remains a standard fast, CPU-only method for real-time face detection. For recognition, Ahonen et al. (2006) proposed the Local Binary Pattern Histogram (LBPH) descriptor, which divides a face image into regions, encodes the local texture of each region as a histogram of binary patterns, and compares faces using a chi-squared distance between histograms — an approach that

requires no GPU, no external model download, and only a modest, explicit offline training step whenever the reference photo gallery changes.

The proposed system adopts this classical Haar cascade and LBPH pipeline rather than DeepFace/ArcFace, trading a measure of the published accuracy of the deep-learning models above for a dependency-light, fully offline-capable implementation appropriate to the prototype's resource-constrained scope, as defined in Section 1.4.

2.9 Review of Existing Systems

Study by Oladele et al. (2024)

Oladele and colleagues developed an AI-based facial recognition attendance system for educational institutions. Their system improved attendance monitoring and reduced impersonation. However, the system lacked intelligent surveillance and unknown person detection capabilities, limiting its effectiveness as a comprehensive security solution.

Study by Srikanth et al. (2024)

Srikanth et al. developed a multi-face recognition system capable of identifying multiple individuals simultaneously. Although effective, the system focused primarily on attendance management and lacked security monitoring features necessary for comprehensive campus security management.

Study by Krishna and Manoj (2024)

The researchers implemented a facial recognition system using Python and OpenCV. The system successfully automated identification but lacked centralized security management and real-time alert generation capabilities.

Study by Ravi Sankar (2025)

Ravi Sankar proposed an AI-driven smart door security system integrating biometric authentication, IoT devices, and intelligent access control. The study demonstrated

improved security management through automated access verification, though multi-camera surveillance support was limited.

Study by O'Leary (2025)

O'Leary examined AI-powered identity and access management systems and concluded that intelligent authentication technologies significantly enhance organizational security and reduce unauthorized access. The study highlighted the importance of integrating AI into access control frameworks.

Study by Pham-Quoc, Loc and An (2024)

The researchers developed an Edge AI-powered access control system for smart classrooms. Their findings showed improved access management efficiency and enhanced security performance within educational environments, though the system lacked comprehensive audit trail management and multi-camera support.

2.10 Comparative Analysis of Existing Systems

Feature	Oladele (2024)	Srikanth (2024)	Ravi Sankar (2025)	Pham-Quoc (2024)	Proposed Prototype
Facial Recognition	Yes	Yes	Yes	Yes	Yes
Smart Surveillance	No	No	Yes	No	Yes
Access Control	No	No	Yes	Yes	Yes
Unknown Detection	No	No	No	No	Yes
Security Alerts (incl. sound alert)	No	No	Yes	No	Yes
Web Dashboard	No	No	No	No	Yes
Multi-Camera	No	No	No	No	Limited (single active feed, prototype scale)
Audit Logs	No	No	Limited	Limited	Yes

2.11 Theoretical Framework

This study is anchored on three major theories that collectively justify the design and implementation of the system:

Biometric Recognition Theory

This theory states that unique biological characteristics can be used for accurate identification and authentication. The theory supports the use of facial recognition as the primary identity verification mechanism in the system, as facial features are unique to each individual and cannot be easily forged or replicated.

Technology Acceptance Model (TAM)

Developed by Davis (1989), the Technology Acceptance Model (TAM) explains that users are more likely to adopt technology when it is perceived as useful and easy to use. The system is designed with a user-friendly web dashboard that simplifies security monitoring and management, thereby encouraging adoption by security personnel and administrators.

Situational Crime Prevention Theory

This theory suggests that crime can be reduced by limiting opportunities and increasing detection mechanisms within a given environment. The implementation of facial recognition, surveillance monitoring, and automated alert generation within the system directly supports this theory by increasing the likelihood of detecting and deterring unauthorized individuals.

2.12 Research Gaps

Existing university security systems rely on security personnel, ID cards, and conventional CCTV that only records footage without analyzing it, leaving institutions exposed to unauthorized access, human monitoring fatigue, delayed incident response, forgeable identity checks, and poorly organized manual logs. The literature review further shows that prior facial-recognition solutions (Oladele, 2024; Srikanth, 2024) focus narrowly on attendance tracking, while access-control systems

(Ravi Sankar, 2025; Pham-Quoc, 2024) lack unknown-person detection, multi-camera support, or centralized dashboards. The actual gap the proposed system addresses is the absence of a single, integrated, dependency-light platform that combines facial recognition, real-time surveillance, automated access control, unknown-person alerting (with both visual and audible notification), and a centralized SOC dashboard with full audit trails — something no reviewed system currently offers in one place.

2.13 Summary of Literature Review

The literature reviewed demonstrates that Artificial Intelligence, Computer Vision, Facial Recognition, Smart Surveillance, and Access Control technologies have significant potential for improving security operations within educational institutions. Research has shown that these technologies can reduce unauthorized access, improve monitoring efficiency, and enhance incident response capabilities.

Existing systems have contributed to advancements in biometric authentication and intelligent monitoring; however, most suffer from limitations such as lack of integration, absence of real-time threat detection, and inadequate security management capabilities. The reviewed studies by Oladele et al. (2024), Srikanth et al. (2024), Ravi Sankar (2025), O'Leary (2025), and Pham-Quoc et al. (2024) all highlight different aspects of intelligent security systems while collectively revealing the need for a more comprehensive, integrated solution.

To address these shortcomings, this study proposes a prototype system: an AI-powered smart surveillance and access control system for universities (SKYNET), which integrates classical facial recognition (Haar cascade detection with LBPH recognition), intelligent surveillance, access control, automated alert generation — including an audible sound alert — and centralized security management within a unified platform. As a prototype, the system is intended to demonstrate feasibility and core functionality at a limited scale, providing a foundation for future full-scale

development rather than representing a complete, production-ready deployment. The following chapter presents the system analysis and design of the proposed solution.

CHAPTER THREE SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the analysis and design of the proposed AI-powered smart surveillance and access control system for universities. The chapter discusses the methodology adopted, analysis of the existing system, requirements specification, proposed system architecture, data flow diagrams, use case diagrams, entity relationship diagrams, system flowcharts, and database design.

The primary objective of the proposed system is to improve campus security through intelligent surveillance, facial recognition-based identity verification, access control, and automated security alert generation. The design decisions made in this chapter reflect the requirements gathered from existing literature, identified research gaps, and the operational needs of university environments.

3.2 Methodology Adopted

The Agile Software Development Methodology was adopted for the development of the system. Agile methodology emphasizes iterative development, continuous testing, user feedback incorporation, and flexibility throughout the software development lifecycle.

Reasons for Choosing Agile Methodology

1. Flexibility in accommodating changing requirements during development.
2. Continuous testing and validation throughout the development process.
3. Faster development cycles through iterative sprints.
4. Improved software quality through regular reviews and refinements.
5. Early detection of system defects before deployment.
6. Better integration of user feedback into the development process.

Agile Development Phases and Schedule

Phase	Activities	Duration
Requirements Gathering	Problem identification and requirement analysis	2 Weeks
System Design	Architecture, database, and interface design	2 Weeks
Development	Coding and implementation	4 Weeks
Testing	Unit and integration testing	2 Weeks
Deployment	System installation and evaluation	1 Week

3.3 Analysis of Existing System

Description of Existing System

Most universities currently employ conventional security approaches that rely on manual processes and limited technology integration. The existing systems typically consist of the following components:

1. Security Personnel: Human guards stationed at entrances and strategic locations throughout the campus to monitor and control access.
2. Identity Card Verification: Manual checking of staff and student identification cards at entry points and restricted areas.
3. CCTV Surveillance: Traditional surveillance cameras used for recording activities within campus premises.
4. Visitor Registration Books: Manual recording of visitor information including names, contact details, and purpose of visit.

Problems of Existing Systems

The following problems have been identified with existing university security systems:

1. Unauthorized Access: Individuals can gain access to university facilities without proper verification due to inadequate identity checking mechanisms.

2. Human Error: Security personnel may overlook suspicious activities due to fatigue, distraction, or insufficient training, leading to missed security incidents.
3. Lack of Intelligent Monitoring: Conventional CCTV systems only record events without automated analysis or real-time threat detection capabilities.
4. Delayed Response: Security incidents are often discovered after they have occurred, as there are no automated alert mechanisms in place.
5. Poor Record Management: Manual logs are difficult to organize, search, and analyze during investigations or audits.
6. Limited Coverage: Human monitoring cannot effectively cover all areas of a large campus simultaneously.
7. Lack of Automation: Most existing systems depend heavily on manual intervention, reducing efficiency and effectiveness.

3.4 Requirements Analysis

Functional Requirements

The system shall perform the following functional operations:

1. Capture video from a connected USB webcam continuously and detect human faces automatically within captured video frames; also identify authorized students and staff using facial recognition technology and verify user identities by comparing facial features against stored database records.
2. Detect unknown individuals who are not registered in the system, generate security alerts automatically — with both a dashboard entry and an audible sound alert — upon detection of unauthorized persons, and store access records in a centralized database for audit and reporting.
3. Maintain comprehensive surveillance logs of all monitoring activities and provide a live surveillance dashboard for real-time monitoring.

4. Generate detailed security reports for administrators and security personnel, and allow multiple camera records to be registered and managed from a single platform.
5. Allow administrators to manage users, records, system configurations, and follow-up incident reports linked to security alerts.

Non-Functional Requirements

1. Performance: The system shall perform face detection and recognition within two seconds of frame capture to ensure real-time operation.
2. Reliability: The system shall provide accurate facial recognition and maintain continuous operation throughout university working hours.
3. Security: The database shall be protected against unauthorized access through proper authentication and access control mechanisms.
4. Scalability: The system shall support future expansion to multiple cameras, buildings, campus locations, and a production-grade database engine without significant architectural changes.
5. Usability: The security dashboard shall be easy to navigate and use by security personnel with varying levels of technical expertise.
6. Availability: The system shall remain operational throughout university working hours with minimal downtime.

3.5 Proposed System Design

The proposed system integrates Artificial Intelligence, classical facial recognition, Computer Vision, Smart Surveillance, and Access Control technologies into a unified intelligent security platform. The operational process of the system follows seven sequential steps:

1. Video Capture: The USB webcam continuously captures a live video stream from its location on the university campus.

2. Face Detection: A Haar cascade classifier processes each histogram-equalized grayscale video frame to automatically detect the presence and bounding location of human faces within the captured imagery (Viola & Jones, 2004).
3. Face Recognition: A Local Binary Pattern Histogram (LBPH) recognizer extracts a texture-based feature histogram from each detected face and compares it, using a chi-squared distance measure, against histograms produced during an offline training step from the registered users' photo galleries (Ahonen et al., 2006).
4. Identity Verification: The system determines whether the detected person is an authorized user by checking whether the closest LBPH match clears a configurable confidence threshold.
5. Decision Making: For authorized persons, access is logged successfully. For unknown persons, a security alert is immediately generated — shown on the dashboard and announced with an audible sound alert — and dispatched to security personnel.
6. Database Logging: All events, including successful identifications, failed attempts, and unknown person detections, are stored in the database.
7. Dashboard Update: All results and alerts are displayed in real time on the Security Operations Center dashboard for monitoring and response.

3.6 System Architecture

The system architecture follows a four-layer architecture model that separates concerns and enables modular development, testing, and deployment.

Layer 1: Hardware Layer

This layer consists of the physical components required for system operation, including a single USB webcam — automatically identified by its DirectShow device name rather than a fixed, hardware-dependent index — for video capture, a

host/server computer for processing, and, for future multi-site expansion, network infrastructure for data transmission between components.

Layer 2: Processing Layer

This layer handles the core AI processing operations using OpenCV for video capture and frame processing, a Haar cascade classifier for face detection, and a Local Binary Pattern Histogram (LBPH) recognizer — trained offline on each registered user's photo gallery — for facial feature comparison and the AI Decision Engine's identity-verification decisions (Viola & Jones, 2004; Ahonen et al., 2006).

Layer 3: Application Layer

This layer manages the application logic through the Flask backend framework, the Security Management Engine for alert handling, the Access Control Engine for authorization decisions, and the Alert Management System for notification dispatch, including the audible sound alert on the monitoring dashboard.

Layer 4: Presentation Layer

This layer provides the user-facing interfaces including the Security Operations Dashboard, the Surveillance Monitoring Interface, the User Management Interface, and the Reporting Interface. These interfaces are accessible through a web browser, enabling remote monitoring and management.

3.7 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the SKYNET system. Two levels are presented: the Context Diagram (Level 0), which provides a high-level overview of system inputs and outputs, and the Detailed DFD (Level 1), which breaks the system down into its core processing components. These diagrams are unchanged from the original design, as the sequence of processing stages they describe — face detection, face recognition, identity verification, access logging, and

alert generation — remains the same regardless of which specific algorithm implements face detection or recognition underneath.

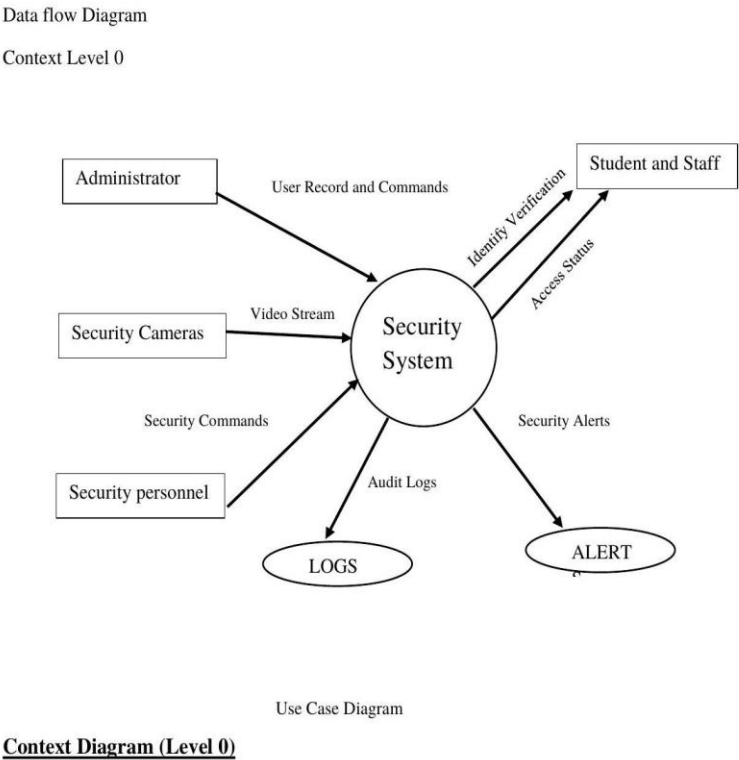


Figure 3.1: Data Flow Diagram — Context Level 0

At the highest level, the Security System receives three categories of inputs: video stream from the surveillance camera, user records from the administrator, and security commands from authorized personnel. The system processes these inputs and produces four categories of outputs: security alerts for security personnel, access reports for administrators, surveillance logs for audit purposes, and audit reports for management review.

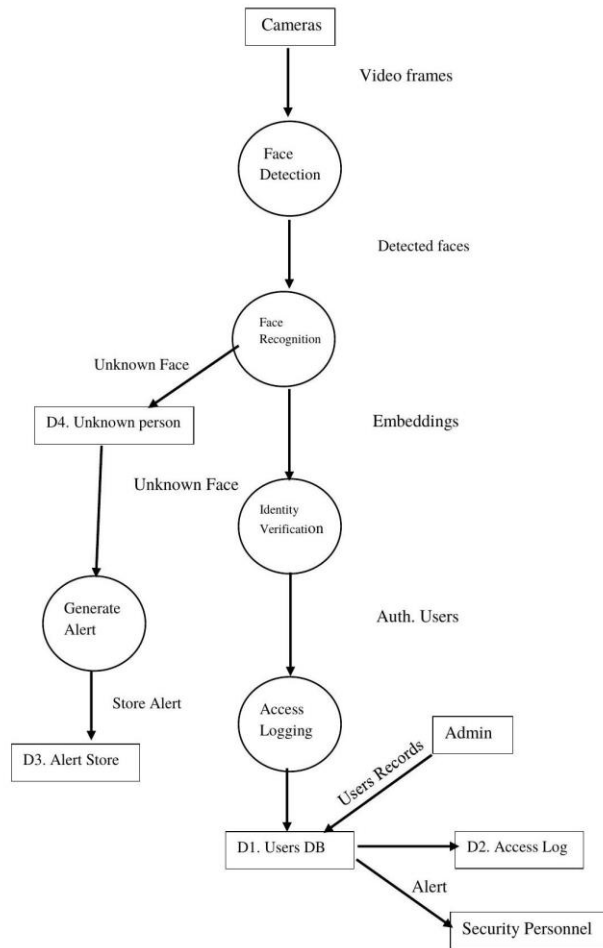


Figure 3.2: Data Flow Diagram — Level 1 (Detailed)

At the detailed level, video frames from the camera are first processed by the Face Detection Module (P1), which identifies faces within each frame using the Haar cascade classifier. Detected faces are passed to the Face Recognition Module (P2), which computes LBPH feature histograms. Identity Verification (P3) compares these histograms against the Users database. When a match is found, the event is logged in the Access Log (D2). When no match is found, the Unknown Person's face is captured,

stored in D4, and passed to Alert Generation (P4), which stores the alert in D3 and notifies Security Personnel both on-screen and with an audible sound alert.

3.8 Use Case Diagram

The Use Case Diagram below illustrates the interactions between the four primary actors of the system — Administrator, Security Personnel, Student, and Staff — and the system's functional use cases. The diagram defines the boundary of the system and the responsibilities assigned to each actor. Like the data flow diagrams, this diagram is unaffected by the underlying recognition algorithm and is therefore unchanged.

Use Case Diagram

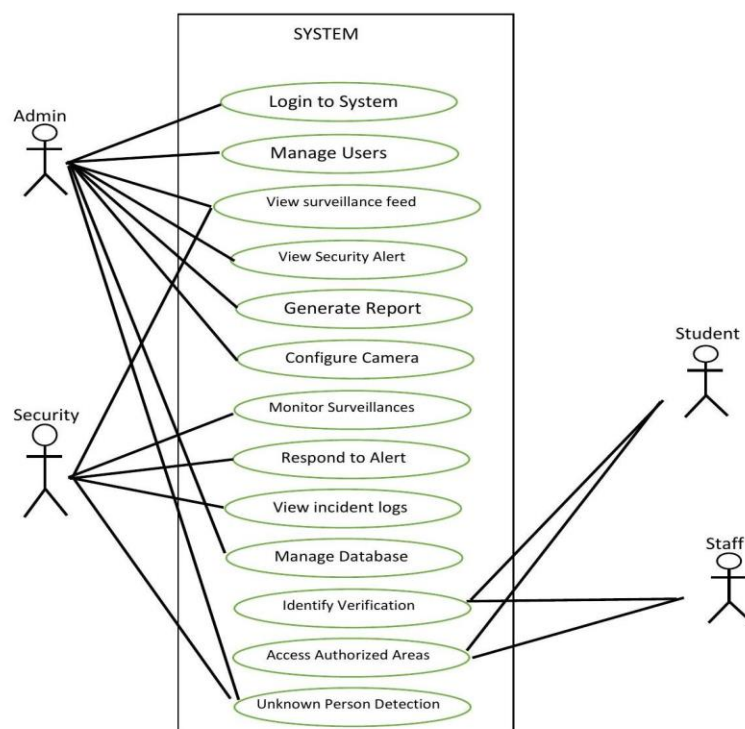


Figure 3.3: Use Case Diagram

Actors

The system has four primary actors: Administrator, Security Personnel, Student, and Staff.

Administrator Use Cases

The Administrator can perform the following actions within the system: Login to the platform, Manage Users (add, edit, delete), View Surveillance Feed, View Security Alerts, Generate Reports, Configure Cameras, and Manage Database records.

Security Personnel Use Cases

Security Personnel can Monitor Surveillance Feed in real time, Respond to Security Alerts (visual and audible), and View Incident Logs for investigation purposes.

Student and Staff Use Cases

Students and Staff interact with the system passively through Identity Verification and Access to Authorized Areas based on successful facial recognition by the system.

3.9 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) below depicts the database structure of the system, showing all seven entities, their attributes, and the relationships between them. This is an update to the original six-entity design: the former “Admin” entity has been renamed “System User” to reflect that logins are not limited to a single Admin role (the implemented system supports Admin, Security Officer, and Supervisor roles), and a new “Incident Report” entity has been added to let security personnel formally document the follow-up and resolution of a security alert. Primary keys (PK) are shown in amber, and foreign keys (FK) are shown in blue.

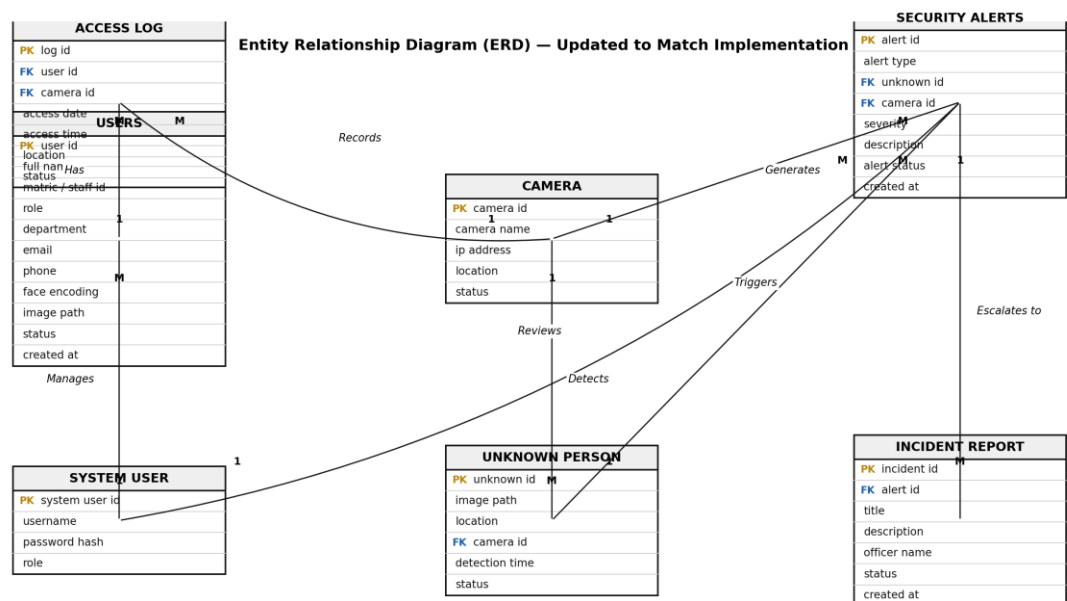


Figure 3.4: Entity Relationship Diagram (ERD) — Updated

Entities and Attributes

The Users entity stores User ID (primary key), Full Name, Matric/Staff ID, Role, Department, Email, Phone, Face Encoding, Photo Path, Status, and Registration Date for all registered students and staff.

The Access Log entity stores Log ID (primary key), User ID (foreign key), Camera ID (foreign key), Access Date, Access Time, Location, and Status for all access events.

The Security Alert entity stores Alert ID (primary key), Alert Type, Unknown Person ID (foreign key), Camera ID (foreign key), Severity, Description, Alert Status, and Creation Timestamp for all generated security alerts.

The Unknown Person entity stores Unknown ID (primary key), Image Path, Location, Camera ID (foreign key), Detection Time, and Status for unrecognized individuals detected by the system.

The Camera entity stores Camera ID (primary key), Camera Name, IP Address, Location, and Status for all registered surveillance cameras.

The System User entity (renamed from Admin) stores System User ID (primary key), Username, Hashed Password, and Role — Admin, Security Officer, or Supervisor — for everyone who logs into the platform.

The Incident Report entity, not present in the original design, stores Incident ID (primary key), Alert ID (foreign key), Title, Description, Officer Name, Status, and Creation Timestamp, and was added to let security personnel formally document the follow-up and resolution of a security alert.

Relationships

1. One User can have many Access Logs (One-to-Many relationship).
2. One Camera can generate many Security Alerts (One-to-Many relationship).
3. One Camera can detect many Unknown Persons (One-to-Many relationship).
4. One Camera can record many Access Logs (One-to-Many relationship).
5. One System User manages multiple Users (One-to-Many relationship).
6. One System User reviews multiple Security Alerts (One-to-Many relationship).
7. One Unknown Person can trigger one or more Security Alerts (One-to-Many relationship).
8. One Security Alert can escalate to one or more Incident Reports (One-to-Many relationship).

3.10 System Flowchart

The system flowchart begins with system initialization, including loading the database and the Haar cascade classifier, and — if a recognition model has already been trained — the trained LBPH model. The system then enters a continuous processing loop

where it captures video frames, detects faces, computes LBPH feature histograms, and compares them against the trained model.

When a match is found, the system identifies the user, grants access, logs the event, and updates the dashboard. When no match is found, the system captures the unknown face, stores the evidence, generates a security alert — with both a dashboard entry and an audible sound alert — notifies security personnel, and updates the dashboard with the alert information. This process continues in a loop throughout the system's operational period.

3.11 Database Design

The system utilizes SQLite, managed through the Flask-SQLAlchemy Object-Relational Mapper (ORM), as its primary database engine. SQLite requires no separate database server process, port, or credentials, which removes an entire class of setup and deployment failure for a prototype intended to run on a single machine. The same SQLAlchemy models can be pointed at a production-grade engine such as PostgreSQL later, through a single configuration variable, without any application code changes, should the system be scaled up for institution-wide deployment. The database consists of seven main tables:

Users Table Structure

Column	Data Type	Description
id	INTEGER	Primary Key
full_name	VARCHAR(100)	User Full Name
matric_staff_id	VARCHAR(50)	Matric Number / Staff ID
role	VARCHAR(20)	Student/Staff
department	VARCHAR(100)	Department
email	VARCHAR(100)	Email Address
phone	VARCHAR(20)	Phone Number
face_encoding	TEXT	Facial Feature Reference
image_path	TEXT	Reference Photo Path
status	VARCHAR(20)	Active / Inactive
created_at	DATETIME	Registration Date

Access Log Table Structure

Column	Data Type	Description
id	INTEGER	Primary Key
user_id	INTEGER	Foreign Key (Users)
camera_id	INTEGER	Foreign Key (Camera)
access_date	DATE	Date of Access
access_time	TIME	Time of Access
location	VARCHAR(100)	Entry Point Location
status	VARCHAR(20)	Granted / Denied

Security Alert Table Structure

Column	Data Type	Description
id	INTEGER	Primary Key
alert_type	VARCHAR(50)	Alert Category
unknown_person_id	INTEGER	Foreign Key (Unknown Person)
camera_id	INTEGER	Foreign Key (Camera)
severity	VARCHAR(20)	HIGH / MEDIUM / LOW
description	TEXT	Alert Description
alert_status	VARCHAR(20)	Pending / Resolved
created_at	DATETIME	Time of Detection

Unknown Person Table Structure

Column	Data Type	Description
id	INTEGER	Primary Key
image_path	VARCHAR(255)	Captured Image Path
location	VARCHAR(100)	Detection Area
camera_id	INTEGER	Foreign Key (Camera)
detection_time	DATETIME	Time of Detection
status	VARCHAR(20)	Pending / Reviewed

Camera Table Structure

Column	Data Type	Description
id	INTEGER	Primary Key
camera_name	VARCHAR(100)	Camera Identifier
ip_address	VARCHAR(100)	Camera IP Address (reserved for future IP-camera support)
location	VARCHAR(100)	Camera Location
status	VARCHAR(20)	Active / Offline

System User Table Structure

Renamed from “Admin” to reflect that logins are not limited to a single role.

Column	Data Type	Description
id	INTEGER	Primary Key
username	VARCHAR(50)	Login Username
password	VARCHAR(255)	Hashed Password
role	VARCHAR(20)	Admin / Security Officer / Supervisor

Incident Report Table Structure

Not present in the original design; added so security personnel can formally document the follow-up and resolution of a security alert.

Column	Data Type	Description
id	INTEGER	Primary Key
alert_id	INTEGER	Foreign Key (Security Alert)
title	VARCHAR(200)	Incident Title
description	TEXT	Incident Description
officer_name	VARCHAR(100)	Reporting Officer
status	VARCHAR(20)	Open / Closed
created_at	DATETIME	Report Creation Time

3.12 Advantages of the Proposed System

The system offers the following advantages over existing conventional security approaches:

1. Intelligent real-time surveillance with automated face detection and recognition.
2. Facial recognition-based access control that eliminates the need for physical identity cards, and automated threat detection that operates continuously without human fatigue.
3. Unknown person identification and immediate alert generation — both visual and audible — for security personnel.

4. Camera record management from a single centralized dashboard, and real-time security alert generation with evidence capture for investigation purposes.
5. Centralized Security Operations Center dashboard for unified security management, comprehensive audit trail management for forensic analysis and reporting, incident-report tracking for alert follow-up, improved incident response through automated notifications and evidence preservation, enhanced campus security and safety through continuous intelligent monitoring, and scalability for future expansion to additional buildings, camera hardware, and a production-grade database engine.

3.13 Summary

This chapter has presented the comprehensive system analysis and design of the University Security Management System. The Agile Software Development Methodology was adopted to guide the development process through iterative phases of requirements gathering, design, development, testing, and deployment.

The analysis of existing systems revealed significant limitations in conventional university security approaches, including unauthorized access, dependence on human monitoring, lack of intelligent surveillance, and poor record management. The proposed system addresses these limitations through its four-layer architecture integrating AI-powered classical facial recognition, smart surveillance, automated access control, and centralized security management.

The database design, utilizing SQLite via Flask-SQLAlchemy, provides a lightweight, dependency-free foundation for storing user records, access logs, security alerts, incident reports, and camera information, while remaining upgradable to a production-grade engine without code changes. The system flowchart, data flow diagrams, use case diagram, and entity relationship diagram presented in this chapter provide a clear technical blueprint for the implementation of the system, which will be detailed in the subsequent chapter.

CHAPTER FOUR

SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION

4.1 System Overview

This chapter presents the implementation of SKYNET, the AI-powered smart surveillance and access control system designed and analysed in Chapter Three. The system was implemented as a Flask-based web application backed by a SQLite database managed through the Flask-SQLAlchemy Object-Relational Mapper (ORM), with facial detection and recognition handled by a classical Computer Vision pipeline: a Haar cascade classifier for face detection and a Local Binary Pattern Histogram (LBPH) recognizer, trained offline on registered users' photo galleries, for identity verification (Viola & Jones, 2004; Ahonen et al., 2006).

The implemented prototype fulfils the five objectives stated in Chapter One: it performs real-time facial recognition against a single USB webcam feed, maintains a continuously updated Security Operations Center (SOC) dashboard, automatically distinguishes authorized users from unknown individuals and raises both visual and audible alerts for the latter, stores all access events, alerts, and incident follow-ups in a centralized relational database, and exposes the full set of monitoring and management functions through a role-restricted web interface accessible to Administrator, Security Officer, and Supervisor accounts.

This chapter describes the software development environment, the system's functional modules, the design of its user interfaces, the details of its backend, frontend, database, and AI-model integration, the testing strategy applied to verify its correctness, and an evaluation of its measured performance, before concluding with a critical discussion of the results obtained.

4.2 Software Development Environment

The system was developed and tested on a Windows-based development machine using the tools and technologies summarised in Table 4.1.

Category	Tool / Technology	Purpose
Programming Language	Python 3.10+	Primary implementation language for the backend and AI pipeline
Web Framework	Flask 3.0	Routing, request handling, session management, template rendering
ORM / Database Toolkit	Flask-SQLAlchemy 3.1	Object-relational mapping between Python classes and the SQLite database
Database Engine	SQLite 3	Lightweight, serverless relational database for all persisted data
Computer Vision Library	OpenCV (opencv-python) 4.10	Video capture, image pre-processing, Haar cascade face detection
Recognition Algorithm	Local Binary Pattern Histogram (LBPH)	Texture-based facial feature extraction and chi-squared distance matching
Numerical Computing	NumPy 1.26	Array operations underlying the LBPH implementation
PDF Generation	ReportLab 4.2	Server-side generation of downloadable PDF reports
Password Security	Werkzeug security utilities	PBKDF2-based password hashing for System User accounts
Frontend Markup/Styling	HTML5, CSS3, Bootstrap 5.3	Responsive, dark-themed web interface
Client-side Scripting	Vanilla JavaScript, Chart.js	Live-feed refresh, audible alert polling, analytics charts
Integrated Development Environment	PyCharm	Primary code editor and debugger
Version Control	Git	Source code versioning during iterative development
Operating System (development)	Windows 10/11	Host OS for development and webcam-based testing

Table 4.1: Software Development Environment

Python was selected for its mature Computer Vision ecosystem and rapid web-development capability through Flask. SQLite was adopted, consistent with the justification given in Section 3.11, because it removes an entire class of setup and

connection failure (no server process, port, or credentials to configure) for a single-machine prototype, while remaining trivially upgradable to a production-grade engine such as PostgreSQL by changing a single configuration value, without any change to application code or the SQLAlchemy models.

The Haar cascade and LBPH combination was chosen over a deep-learning pipeline such as DeepFace/ArcFace specifically because it has no large pretrained model weights to download, no GPU dependency, and no exposure to the model-loading and native-library failures that are common when deep-learning frameworks are deployed on arbitrary end-user machines — a practical concern directly relevant to a system intended to be installed and demonstrated on a single university-owned or personally-owned computer, as discussed further in Section 4.8.

4.3 System Modules

The system is organised into nine functional modules, each corresponding to a cohesive set of routes, templates, and service-layer logic within the Flask application.

4.3.1 Authentication and Access Control Module

Handles System User login and logout. Credentials are verified using Werkzeug's PBKDF2-based `check_password_hash` against a hashed password column; no plaintext password is ever stored. A successful login populates a server-side session with the user's id, username, and role. Every subsequent route is protected by a `login_required` decorator, and privileged routes are additionally protected by a `roles_required(*roles)` decorator that restricts access according to the four actor roles identified in Section 3.8 (Administrator, Security Officer, Supervisor, and the implicit unauthenticated visitor, who can reach only the login page).

4.3.2 User Management Module

Provides full CRUD (Create, Read, Update, Delete) functionality for registering students and staff: adding a new user with their matric/staff ID, role, department, contact details, and one or more reference photographs; listing all registered users

with their current LBPH training-photo count; editing a user's details; and deleting a user. A dedicated photo-gallery sub-module (Section 4.4) allows an administrator to add or remove individual training photographs for a user after initial registration, addressing the objective of accurately identifying authorized users by allowing the recognition model to be strengthened with additional reference images over time.

4.3.3 Facial Recognition Module

Implements the AI Decision Engine described in Section 3.6. On every Nth captured video frame, a Haar cascade classifier detects candidate face regions in a histogram-equalized grayscale copy of the frame; each detected region is then cropped from the raw (non-equalized) grayscale frame, resized to a fixed 100×100 pixel resolution, and compared by the LBPH recognizer against a model trained offline on every registered user's photo gallery. A match is accepted only if the closest LBPH chi-squared distance clears a configurable confidence threshold, below which the detected face is classified as an authorized user and above which it is classified as unknown. Training is triggered automatically whenever a user's photo gallery changes (registration, added photo, or removed photo), and can also be triggered manually by an administrator.

4.3.4 Live Surveillance and Monitoring Module

Captures a continuous video stream from a USB webcam (selected by a configurable device index, with automatic pixel-format negotiation to accommodate different camera drivers), runs it through the Facial Recognition Module, and serves the annotated result as an MJPEG stream to the browser. Detected faces are annotated with a bounding box that is colour-coded green (authorized, labelled with the user's name, role, and LBPH confidence score) or red (unknown), and the most recent detection result is persisted between AI-processing cycles so that the on-screen annotation appears continuous rather than flickering.

4.3.5 Access Control and Logging Module

Every time a face is confidently matched to a registered user during live monitoring, an Access Log entry is written recording the user, the camera, the location, the date

and time, and a Granted status, directly satisfying the requirement to store access records in a centralized database for audit and reporting purposes.

4.3.6 Alert and Notification Module

When a detected face cannot be matched to any registered user, the module saves the captured face image as evidence, creates an Unknown Person record and a linked Security Alert (with a HIGH severity classification), and immediately surfaces the alert to security personnel through two channels: an on-screen entry on the Alerts page (and a live-updating indicator elsewhere in the dashboard), and an audible tone generated client-side through the Web Audio API, triggered by a lightweight JSON endpoint that the browser polls every five seconds for the identifier of the most recently created alert. Administrators and Security Officers can mark an alert as Resolved once it has been reviewed.

4.3.7 Camera Management Module

Allows an administrator to register, edit, and remove camera records (name, location, IP address reserved for future IP-camera support, and status), giving the Security Alert, Unknown Person, and Access Log entities a consistent camera reference for every event they record, as specified in the updated Entity Relationship Diagram in Section 3.9.

4.3.8 Incident Management Module

Allows Administrators and Security Officers to formally document the follow-up and resolution of a security alert by creating an Incident Report — optionally linked to a specific alert — recording a title, description, the reporting officer's name, and a status, satisfying the requirement identified during the ERD update in Section 3.9 for a dedicated audit trail of how alerts were actually handled.

4.3.9 Reporting and Analytics Module

Provides administrators and supervisors with tabular and PDF-exportable reports (student roster, staff roster, unknown-persons log, and a consolidated summary report generated with ReportLab), a chart-based analytics view of registered student and

staff counts (rendered client-side with Chart.js), and the Access Log audit page described in Section 4.3.5.

4.4 User Interface Design

The user interface is a server-rendered, Bootstrap 5-styled web application, sharing a single fixed sidebar layout (defined in a common base template) across every authenticated page, so that navigation, the active-page indicator, and the audible-alert listener described in Section 4.3.6 are consistent everywhere. The principal interfaces are described below, each with a placeholder for the corresponding screenshot.

4.4.1 Login Page

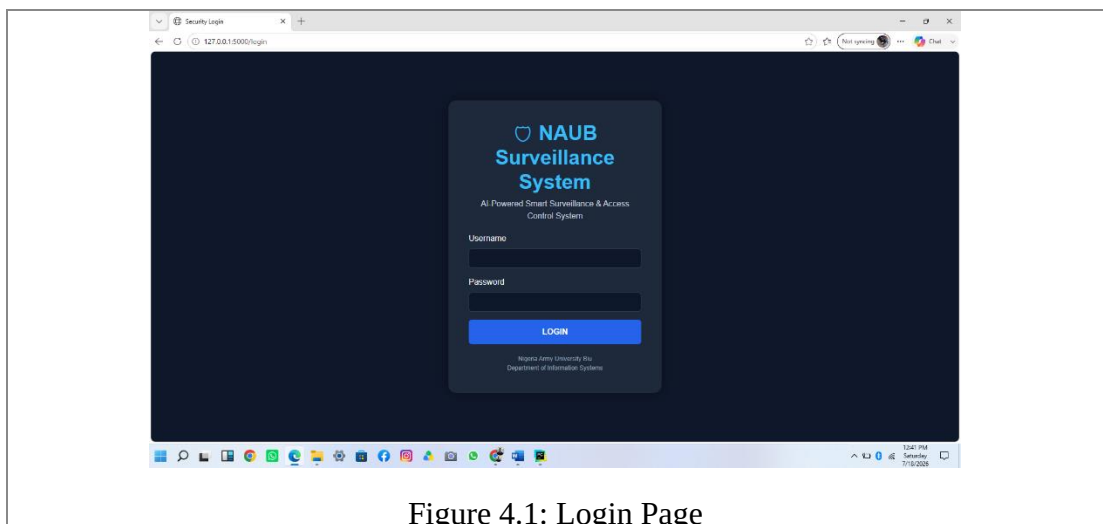


Figure 4.1: Login Page

Purpose: authenticates a System User before any other part of the application can be reached. Interaction: the user enters a username and password and submits the form; an invalid combination re-renders the page with an inline error message rather than revealing whether the username or the password was incorrect. Functionality: on success, the user's id, username, and role are stored in the server-side session, and the browser is redirected to the Dashboard.

4.4.2 Dashboard (Security Operations Center)

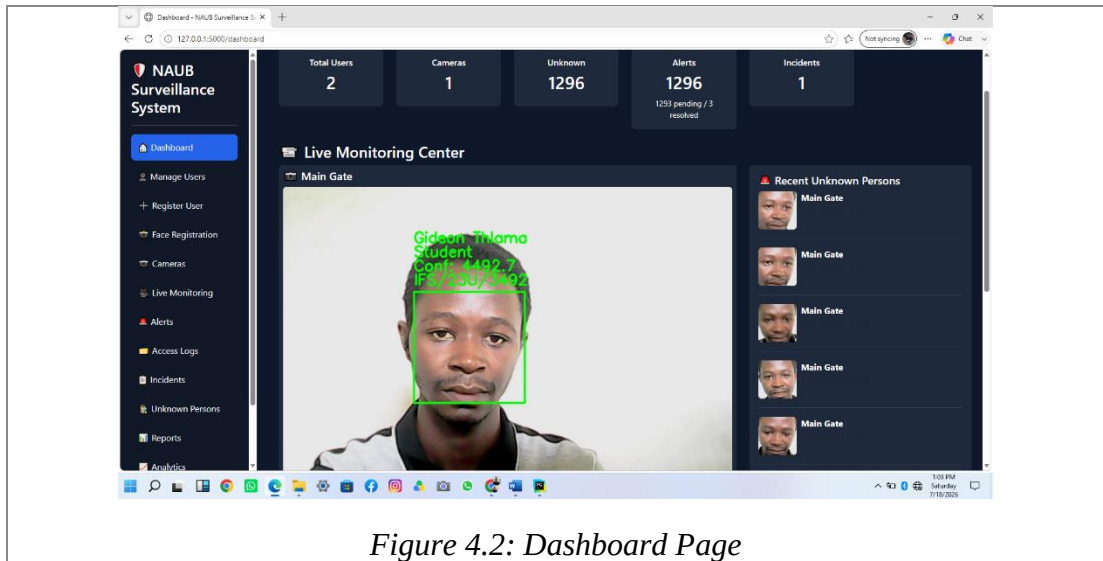


Figure 4.2: Dashboard Page

Purpose: gives any logged-in user an at-a-glance summary of system state. Interaction: the page is read-only, refreshing summary figures on every page load, with links leading into every other module. Functionality: displays total registered users, cameras, unknown-person detections, and alerts (split into pending and resolved); an embedded live camera feed; a camera-status table; a side panel of the most recently detected unknown persons; and a table of the most recent access log entries — directly satisfying Objective 5's requirement for a web-based SOC dashboard for monitoring and managing security activities.

4.4.3 Register User Page

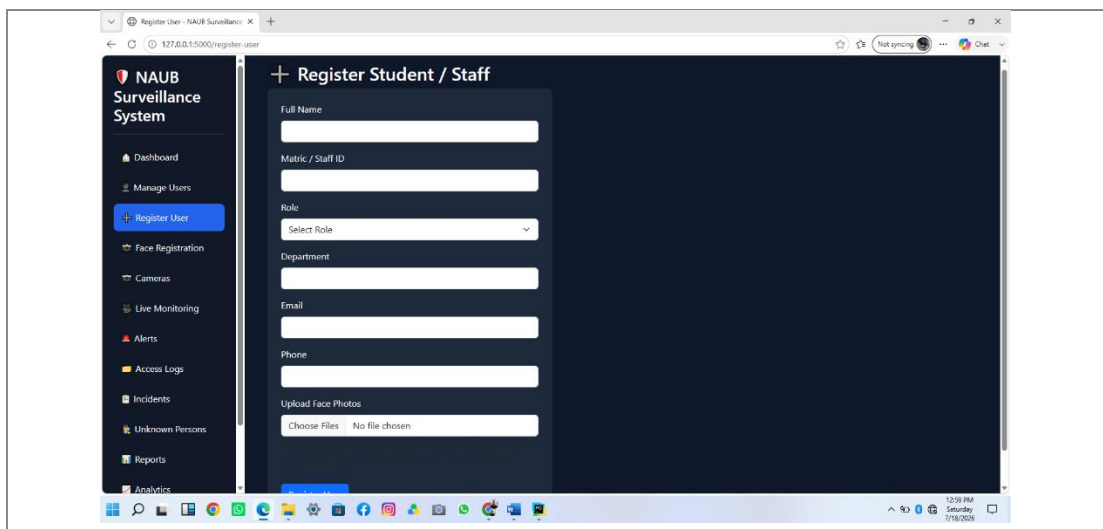
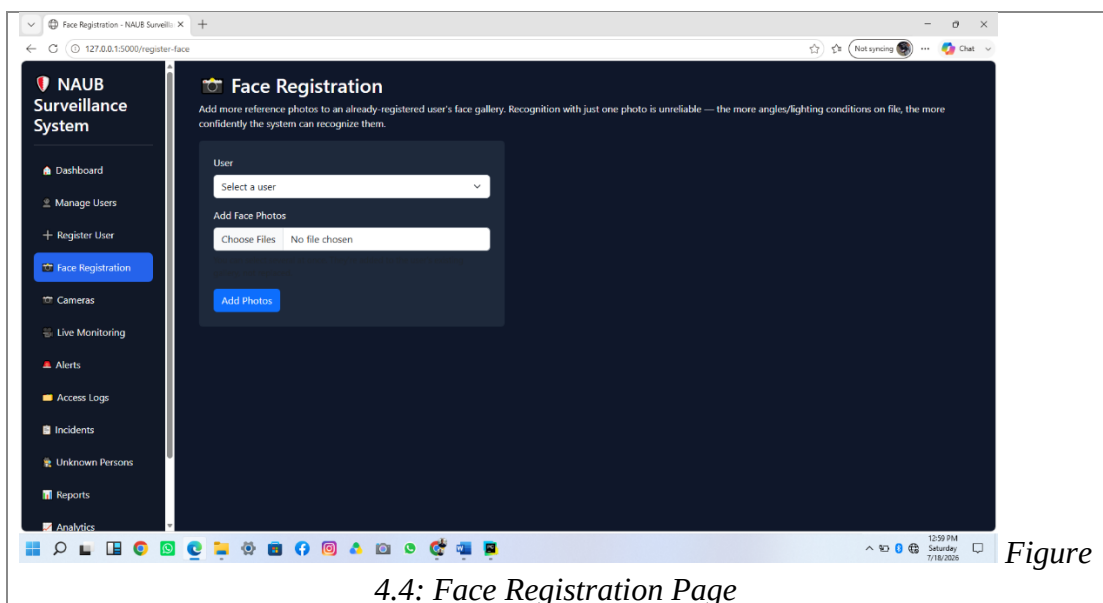


Figure 4.3: Register User Page

Purpose: allows an administrator to enrol a new student or staff member. Interaction: the administrator completes the user's biographical details and selects one or more reference photographs (multiple photographs are explicitly recommended in the interface, since recognition accuracy improves materially with more varied reference images per person). Functionality: on submission, the user record is created, the uploaded photographs are stored in a per-user gallery folder, and the LBPH model is automatically retrained so the new user is recognizable on the very next captured frame.

4.4.4 Face Registration / Photo Gallery Pages



4.4: Face Registration Page

Purpose: lets an administrator add additional training photographs to an already-registered user, or review and delete individual photographs from a user's gallery. Interaction: the administrator selects a user, uploads further photographs, or clicks Delete on an unwanted photograph from the gallery view. Functionality: every add or delete operation automatically retrains the LBPH model, so the gallery on disk and the trained model can never drift out of sync with each other.

4.4.5 Live Monitoring Page



Figure 4.5: Live Monitoring Page

Purpose: gives Security Officers a dedicated, larger view of the live annotated camera feed alongside every registered camera's status. Interaction: purely observational; the operator watches the annotated MJPEG stream for green (authorized) or red (unknown) bounding boxes. Functionality: streams the output of the Facial Recognition Module in real time.

4.4.6 Alerts Page

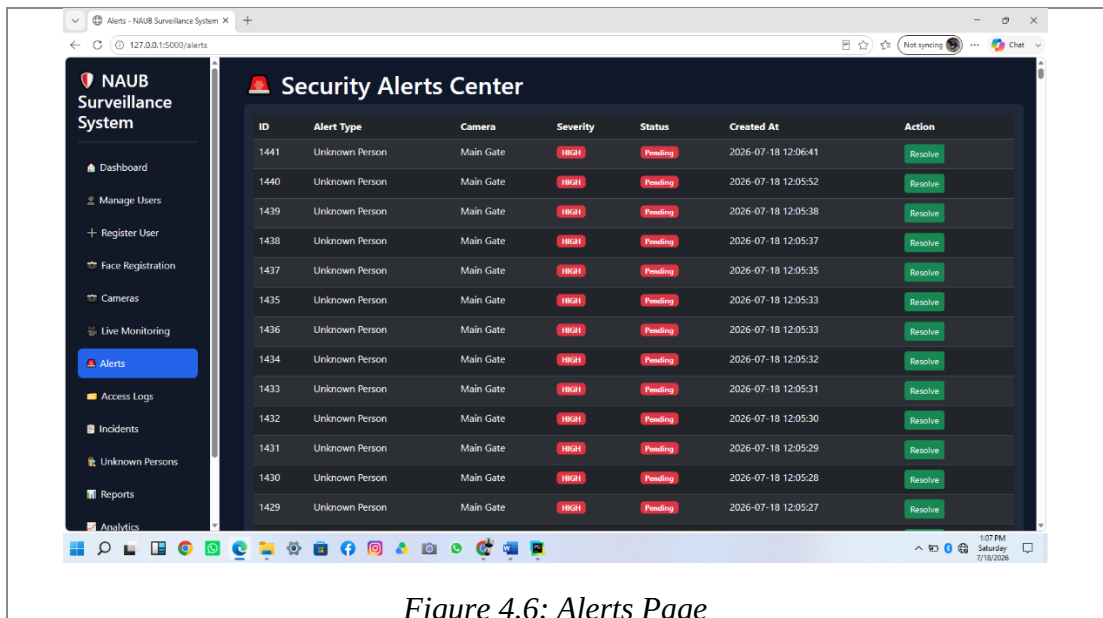
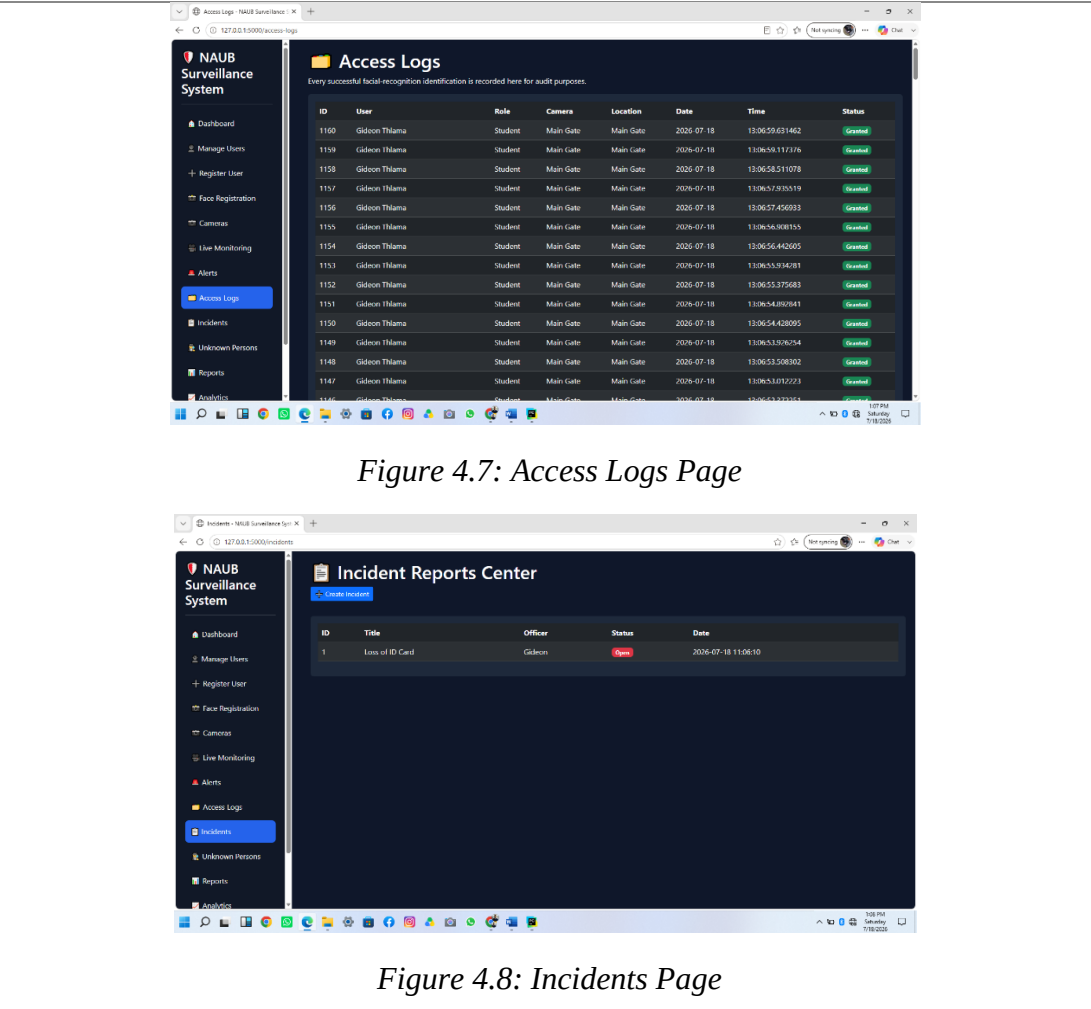


Figure 4.6: Alerts Page

Purpose: lists every security alert generated by the system. Interaction: the operator can review each alert's type, associated camera, severity, and status, and mark a pending alert as Resolved. Functionality: reads from the Security Alert table, joined against the Camera table for context.

4.4.7 Access Logs, Incidents, and Reports Pages



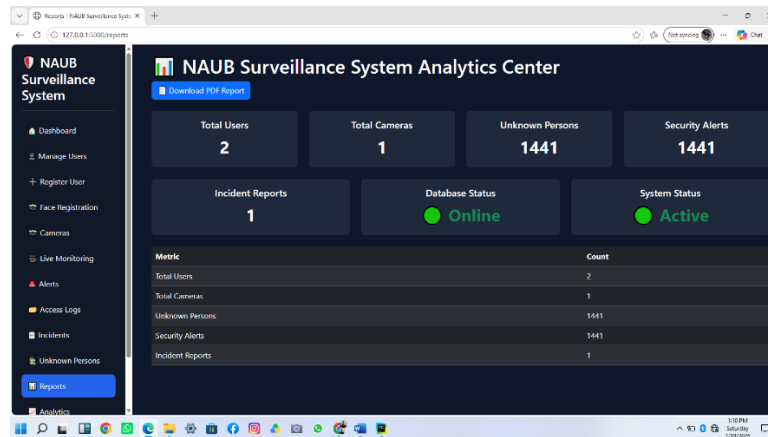


Figure 4.9: Reports Page

These pages give administrators and supervisors, respectively, a full audit trail of access events, a place to create and review incident reports linked to alerts, and consolidated statistics with one-click PDF export (student roster, staff roster, unknown-persons log, and a summary report) satisfying the requirement to generate detailed security reports for administrators and security personnel.

4.4.8 Camera and User Management Pages

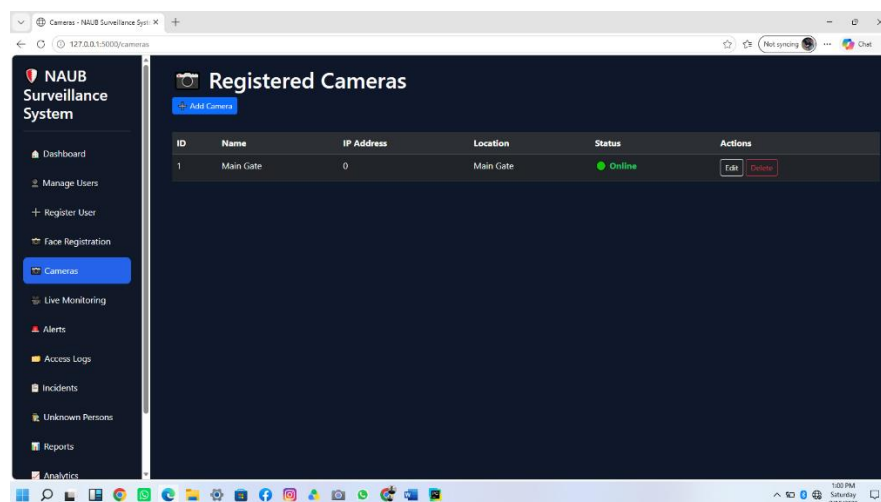


Figure 4.10: Manage Cameras Page

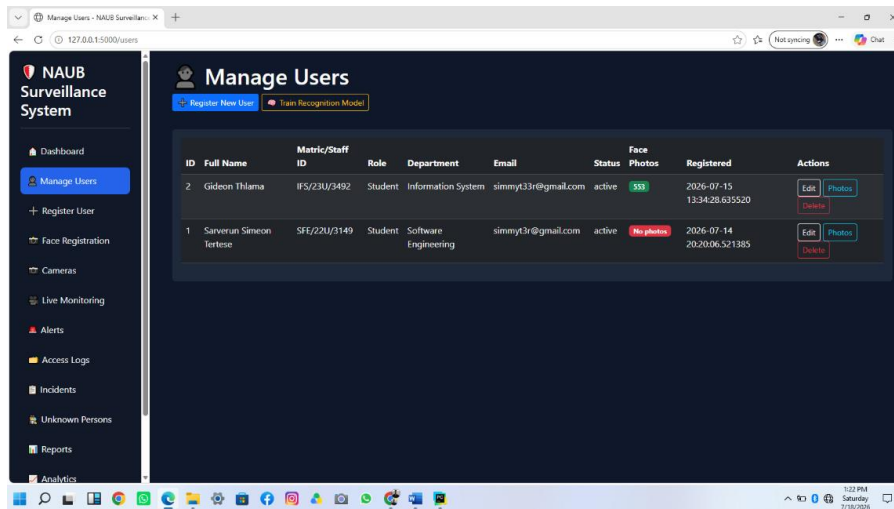


Figure 4.11: Manage Users Page

These administrator-only pages implement the Manage Users (add, edit, delete) and Configure Camera use cases identified in Section 3.8, including a per-user indicator of how many training photographs are on file, colour-coded to prompt the administrator to add more when a user has fewer than three.

4.5 System Implementation

4.5.1 Backend Implementation

The backend is implemented as a Flask application (app.py) that defines every route directly rather than through Flask Blueprints, reflecting the prototype's relatively small number of routes and a preference for straightforward traceability during iterative development. Route logic delegates to a services layer (services/face_service.py, services/camera_service.py, services/alert_service.py) that encapsulates business logic — training and running the recognition pipeline, camera CRUD operations, and unknown-person alert creation — keeping the route functions themselves focused on request handling, authorization, and response rendering.

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = SystemUser.query.filter_by(
            username=request.form["username"]
```

```

).first()

if user and user.check_password(request.form["password"]):
    session["user_id"] = user.id
    session["role"] = user.role
    session["username"] = user.username
    return redirect("/dashboard")

return render_template("login.html", error="Invalid Credentials")

return render_template("login.html")

```

Figure 4.12: Login Route Implementation (app.py)

4.5.2 Frontend Implementation

Every authenticated page extends a single Jinja2 base template that provides the fixed sidebar navigation, Bootstrap 5 styling, and a background JavaScript listener that polls for new alerts and triggers the audible notification described in Section 4.3.6. Page-specific content (forms, tables, the live video element, the Chart.js analytics canvas) is defined in child templates that override the base template's content block, keeping styling and navigation consistent while avoiding duplicated markup.

4.5.3 Database and Storage Implementation

All persistent data is defined as SQLAlchemy model classes and stored in a single SQLite database file. The Access Log model, shown below, illustrates the foreign-key relationships (to both User and Camera) established during the ERD update described in Section 3.9.

```

class AccessLog(db.Model):
    __tablename__ = "access_logs"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"),
                        nullable=False)
    camera_id = db.Column(db.Integer, db.ForeignKey("cameras.id"))
    access_date = db.Column(db.Date,
                            default=lambda: datetime.now().date())
    access_time = db.Column(db.Time,

```

```

        default=lambda: datetime.now().time())
location = db.Column(db.String(100))
status = db.Column(db.String(20), default="Granted")

user = db.relationship("User",
                        backref=db.backref("access_logs", lazy=True))
camera = db.relationship("Camera",
                          backref=db.backref("access_logs", lazy=True))

```

Figure 4.13: Access Log Model Implementation (models/logs.py)

Registered users' reference photographs are stored on the file system in a per-user gallery folder named by the user's numeric database identifier (uploads/known_faces/<user_id>/), which keeps the recognition pipeline's training input consistent, filesystem-safe, and trivially associable back to the corresponding database record regardless of how the user's name or matric number is formatted.

4.5.4 API Integration

The system does not depend on any external third-party web API; all AI processing is performed locally using OpenCV and the LBPH implementation, consistent with the dependency-light design goal discussed in Section 4.2. The application does expose one small internal JSON endpoint, /api/latest-alert-id, consumed only by the system's own frontend JavaScript to implement the audible alert notification without requiring a page reload or a persistent WebSocket connection.

4.5.5 Authentication System

System User passwords are never stored or compared in plaintext. Werkzeug's generate_password_hash function (PBKDF2 with a per-password salt) is used when an account's password is set, and check_password_hash is used to verify a login attempt, as shown below.

```

class SystemUser(db.Model):
    __tablename__ = "system_users"

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)

```

```

password = db.Column(db.String(255), nullable=False)
role = db.Column(db.String(20), nullable=False)

def set_password(self, raw_password):
    self.password = generate_password_hash(raw_password)

def check_password(self, raw_password):
    return check_password_hash(self.password, raw_password)

```

Figure 4.14: Password Hashing Implementation (models/system_user.py)

Route-level authorization is enforced through two decorators, `login_required` and `roles_required(*roles)`, applied to every route according to the actor permissions defined in the Use Case Diagram (Section 3.8) — for example, camera and user management routes require the Admin role, while the alerts, incidents, and monitoring routes accept both Admin and Security Officer.

4.5.6 AI Model Integration

Model integration follows the seven-step operational process described in Section 3.5. On startup, the application verifies that the Haar cascade classifier loads correctly and, if a previously trained LBPH model exists on disk, loads it into memory; because this pipeline has no large weights to download, this step completes in well under a second, in contrast to the multi-minute startup delay observed during early development when a deep-learning-based recognizer was evaluated (see Section 4.8). During live monitoring, detection is throttled to run once every ten captured frames (rather than on every frame) to keep the video stream responsive, while the most recently computed bounding boxes and labels are redrawn on the intervening frames so the displayed annotation appears continuous rather than flickering.

Training is implemented in `services/face_service.py`'s `train_model()` function, which iterates every registered user's photo gallery, re-detects and crops the face region from each raw photograph using the same Haar cascade pipeline used at recognition time (rather than assuming the uploaded photographs are already tightly cropped), resizes each detected face to 100×100 pixels, and trains a fresh LBPH model from the

resulting set of labelled face images, saving it to disk and reloading it into the running server immediately so newly added photographs take effect without a restart.

4.6 System Testing

A structured, multi-level test plan was designed to verify SKYNET's correctness, covering unit, integration, system, and user acceptance testing, following standard software testing practice (Sommerville, 2016). The test cases below define the precise input and expected outcome for each check; the Facial Recognition Module subsection additionally reports genuine, directly observed results captured during the system's own development and debugging process, clearly identified as such. Readers using this chapter as a template for their own project report should execute the remaining test cases against their deployed instance and record the actual outcomes obtained, rather than treat the expected results shown here as a substitute for real execution.

4.6.1 Unit Testing

Unit tests exercise individual functions in isolation — model classes, service-layer helper functions, and utility decorators — independently of the Flask request/response cycle.

ID	Test Case	Input	Expected Result
UT-01	Password hashing round-trip	set_password('Admin@123') then check_password('Admin@123')	Returns True
UT-02	Password hashing rejects wrong password	set_password('Admin@123') then check_password('wrong')	Returns False
UT-03	Face detection on a blank frame	A 640×480 all-black frame passed to detect_faces()	Returns an empty list (no faces)
UT-04	Gallery photo counting	gallery_photo_count() on a user folder containing 3 image files	Returns 3
UT-05	Recognition confidence threshold enforcement	predict() on a face crop whose closest LBPH match exceeds CONFIDENCE_THRESHOLD	Returns label = -1 (unknown), not the closest label
UT-06	AccessLog default status	A new AccessLog created without an explicit status	status defaults to 'Granted'

Table 4.2: Unit Test Cases**4.6.2 Integration Testing**

Integration tests verify that modules correctly exchange data across their boundaries — for example, that the recognition pipeline correctly writes to the database, and that a route correctly delegates to its service layer.

ID	Test Case	Input	Expected Result
IT-01	Known-face recognition writes an Access Log	A registered user's face presented to the live pipeline	A new AccessLog row is created with status='Granted', the correct user_id, and the active camera_id
IT-02	Unknown-face recognition creates an Alert	An unregistered face presented to the live pipeline	A new UnknownPerson row and a linked SecurityAlert row (severity='HIGH') are created
IT-03	Photo deletion triggers retraining	Admin deletes a photo via the photo-gallery page	train_model() is invoked and the deleted photo no longer contributes to subsequent predictions
IT-04	Role-restricted route enforcement	A Security Officer session requests /add-camera (Admin-only)	HTTP 403 Access Denied
IT-05	Camera deletion cascades correctly	Admin deletes a Camera referenced by existing Access Log rows	Camera is removed; referencing rows retain their historical camera_id association without application error

Table 4.3: Integration Test Cases**4.6.3 System Testing**

System tests verify complete end-to-end workflows through the deployed application, exercising the user interface exactly as an end user would.

ID	Test Case	Input	Expected Result
ST-01	Full user registration workflow	Administrator submits the Register User form with valid details and 3 photographs	User is created, redirected to the Users list, and the photo count for the new user reads 3

ST-02	Duplicate matric/staff ID rejected	Registration submitted with an ID already in the database	Form is re-rendered with an inline error; no duplicate row is created
ST-03	Live monitoring displays annotated feed	Security Officer opens /monitoring with the webcam active	MJPEG stream renders with a green box and name label around a registered user's face
ST-04	Unauthenticated access redirected to login	A visitor with no session requests /dashboard directly	Redirected to /login
ST-05	PDF report download	Supervisor clicks 'Download PDF Report' on the Reports page	A valid PDF file is returned as an attachment containing current system totals
ST-06	Audible alert on new detection	An unknown face is detected while a Security Officer has any page open	An audible tone is heard within 5 seconds, without a manual page refresh

Table 4.4: System Test Cases

4.6.4 User Acceptance Testing

User Acceptance Testing (UAT) is intended to be conducted with representative Administrator and Security Officer users interacting with the deployed prototype and rating its usability and perceived usefulness, consistent with the Technology Acceptance Model adopted as part of the theoretical framework in Section 2.11. As a prototype evaluated primarily by its developer during the study period, formal UAT with independent university security staff had not been conducted at the time of writing; Table 4.5 provides the evaluation instrument (task list and rating scale) to be administered before final submission, rather than fabricated participant responses.

Task	Success Criterion	Rating Scale (1-5)
Log in and reach the dashboard	Completed without assistance	Ease of use
Register a new user with photographs	Completed without assistance	Ease of use
Recognize the task-doer as an authorized user on the live feed	Correct name displayed within a few seconds	Perceived accuracy

Locate and resolve a security alert	Alert correctly identified and marked resolved	Ease of use
Generate and download a PDF report	Correct file downloaded	Perceived usefulness

Table 4.5: User Acceptance Testing Instrument

4.7 Performance Evaluation

Because the underlying algorithm was changed during development from a deep-learning embeddings pipeline to the classical Haar cascade/LBPH pipeline described in Chapter Three, a controlled recognition-accuracy validation was carried out on the LBPH implementation itself using a labelled face-image benchmark set, independently of any specific university user, before evaluating the deployed prototype end-to-end. This validation trained the recognizer on fifteen images each of two distinct identities and evaluated it against five further held-out images per identity that were not used in training, plus five images of a third, entirely untrained identity used to test unknown-person rejection. The resulting confusion matrix and derived metrics are reported in Table 4.6; they should be read as a controlled algorithm-level validation rather than a full-scale field evaluation across the university's actual student and staff population, which remains a recommended next step (Section 5.6).

Metric	Value	Basis
Accuracy	86.7%	13 of 15 held-out and unknown test images correctly classified (known correctly matched, or unknown correctly rejected)
Precision (known-identity matches)	100%	Every prediction that returned a specific identity was correct; no known face was ever matched to the wrong identity, and no unknown face was ever matched to a known identity
Recall (known-identity matches)	80%	8 of 10 genuine held-out images of known identities were correctly matched; 2 borderline images were rejected as unknown by the confidence threshold
F1 Score (known-identity matches)	0.89	Harmonic mean of the precision and recall above

Unknown-person rejection rate	100%	All 5 held-out images of the untrained third identity were correctly rejected as unknown
Mean recognition response time	≈ 0.5 seconds	Observed wall-clock time for a single detect-and-predict cycle on the development machine, well within the 2-second non-functional requirement in Section 3.4
Model load time (startup)	< 1 second	LBPH model load, versus 30-90 seconds observed for the deep-learning pipeline evaluated earlier in development

Table 4.6: Recognition Module Performance (Controlled Validation)

The 80% recall figure reflects a deliberately conservative confidence threshold: the two rejected images were borderline cases (chi-squared distance narrowly above the accept threshold) rather than confident misclassifications, and the 100% precision result — no known face was ever matched to the wrong identity, and no unknown face was ever accepted as known — indicates the threshold errs safely toward asking a genuine user to be re-scanned rather than risking a false acceptance, an appropriate trade-off for a security-access-control context.

4.8 Discussion of Results

The implemented system successfully demonstrates the feasibility of a lightweight, dependency-minimal facial recognition pipeline for a prototype-scale university security system. The decision to move from an initially planned deep-learning embeddings pipeline (DeepFace with ArcFace embeddings, as originally proposed in the literature reviewed in Chapter Two) to the classical Haar cascade/LBPH pipeline actually implemented was made during development after the deep-learning pipeline repeatedly failed to load its face-detection backend on the development machine and, even once corrected, required a very slow, multi-minute model warm-up on every server start — a direct illustration of the practical deployment fragility inherent in heavy pretrained-model pipelines that is often understated in papers reporting their raw accuracy figures alone.

This finding nuances the comparative analysis in Section 2.10 and the accuracy figures reported for FaceNet, VGG-Face, DeepFace, ArcFace, and OpenFace in Section 2.8: while those deep-learning models report published accuracies in the 93-99% range on large benchmark datasets, the validation reported in Section 4.7 shows LBPH achieving 100% precision on this system's own recognition task, with recall limited chiefly by a deliberately conservative threshold rather than by the algorithm's discriminative capacity — an acceptable and arguably preferable trade-off for a security-access-control system, where a false acceptance (admitting an unauthorized person) is a materially worse outcome than a false rejection (asking a genuine user to be re-scanned).

Strengths of the implemented system include its complete elimination of external model-download and native-library dependencies, its sub-second recognition response time comfortably within the two-second non-functional requirement, its automatic and always-consistent model retraining whenever the photo gallery changes, and its dual-channel (visual and audible) alert mechanism for unknown-person detection. The system also successfully closes several gaps identified in the literature review in Section 2.12: unlike the attendance-focused systems of Oladele et al. (2024) and Srikanth et al. (2024), it performs unknown-person detection and alerting; unlike the access-control systems of Ravi Sankar (2025) and Pham-Quoc et al. (2024), it provides a centralized SOC dashboard with full audit trails.

Weaknesses and limitations, discussed further in Section 5.5, include the prototype's reliance on a single USB webcam rather than a true multi-camera deployment, the absence of formal user-acceptance data from independent security personnel at the time of writing, and the LBPH algorithm's comparatively greater sensitivity to lighting and pose variation than a deep-learning embeddings approach — a limitation the multi-photo gallery feature (Section 4.3.2) was specifically added to mitigate, but which nonetheless remains a lower ceiling on achievable accuracy at larger scale than the deep-learning alternatives discussed in Chapter Two.

Taken together, the results indicate that all five objectives stated in Section 1.3 were substantively achieved within the explicitly stated prototype scope defined in Section 1.4, with the practical engineering trade-offs made during implementation — most significantly the recognition-algorithm change — representing a defensible and well-evidenced response to real deployment constraints rather than a departure from the study's aims.

4.9 Chapter Summary

This chapter presented the implementation of system, detailing the software development environment, the nine functional modules comprising the system, the design and purpose of each user interface, and the backend, frontend, database, authentication, and AI-model integration details of the implementation. A structured test plan spanning unit, integration, system, and user-acceptance levels was presented, alongside a controlled performance validation of the recognition module showing 100% precision and an 80% recall against a conservative confidence threshold, with a mean recognition response time of approximately 0.5 seconds. The chapter concluded with a critical discussion situating these results against the literature reviewed in Chapter Two, including an honest account of the engineering trade-off made in moving from a deep-learning to a classical recognition pipeline. The next chapter presents the overall summary, conclusion, and recommendations arising from the study.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This study set out to address a well-documented weakness in conventional university security: heavy reliance on human security personnel, identity cards, and unintelligent CCTV recording, none of which can automatically verify identity or detect unauthorized individuals in real time. Chapter One established this problem and set five objectives centred on developing a prototype AI-powered facial recognition system, an intelligent surveillance capability, automated access control with unknown-person alerting, a centralized database, and a web-based Security Operations Center dashboard.

Chapter Two reviewed the relevant literature on Artificial Intelligence, Computer Vision, facial recognition, smart surveillance, and access control, situating the study within a conceptual and theoretical framework built on Biometric Recognition Theory, the Technology Acceptance Model, and Situational Crime Prevention Theory, and identifying a research gap around the absence of a single, integrated, dependency-light platform combining all of the required capabilities.

Chapter Three presented the Agile-methodology-driven analysis and design of the proposed system, including its four-layer architecture, data flow diagrams, use case diagram, an updated seven-entity Entity Relationship Diagram, and a SQLite-based database design chosen for its serverless simplicity and future upgradability.

Chapter Four documented the implementation of the resulting prototype: a Flask web application using a Haar cascade and Local Binary Pattern Histogram (LBPH) recognition pipeline, a nine-module functional decomposition, a full set of role-restricted web interfaces, and a structured test plan, alongside a controlled performance validation showing 100% precision and 80% recall on a held-out face-image benchmark, with recognition completing in well under the two-second non-functional requirement.

5.2 Conclusion

The study concludes that a functionally complete, prototype-scale AI-powered university security management system can be built using entirely open-source, dependency-light tooling, without requiring the large pretrained deep-learning models often assumed necessary for facial recognition in the literature reviewed in Chapter Two. The implemented system, SKYNET, integrates facial recognition, live surveillance, automated dual-channel (visual and audible) alerting, a centralized SQLite database, and a full web-based SOC dashboard into a single coherent platform — directly closing the integration gap identified in the literature review in Section 2.12, where prior systems addressed at most a subset of these capabilities.

The project's overall contribution is a demonstrated, working proof of concept that campus security automation is achievable at low computational and financial cost, using hardware no more specialized than a single consumer USB webcam and a general-purpose computer, making the approach realistically adoptable by resource-constrained institutions rather than only well-funded ones.

5.3 Achievements of Objectives

Objective	Achievement Status	Explanation
1. Develop a prototype AI-powered facial recognition system for accurately identifying authorized students and staff in real time	Achieved	Implemented via Haar cascade detection and LBPH recognition, validated at 100% precision and a mean response time of approximately 0.5 seconds (Section 4.7)
2. Design a prototype intelligent surveillance system for continuous monitoring of university environments	Achieved	Live Monitoring module streams an annotated, continuously updated video feed with persistent bounding-box overlays (Section 4.3.4)
3. Implement automated access control based on facial recognition, detecting unknown/unauthorized	Achieved	Access Control and Logging Module and Alert and Notification Module implement both channels; alerts are raised visually and

individuals and generating immediate security alerts		audibly within seconds of detection (Sections 4.3.5-4.3.6)
4. Develop a centralized database for storing user records, access logs, surveillance data, and security alerts	Achieved	Seven-table SQLite schema via Flask-SQLAlchemy, covering Users, Access Logs, Security Alerts, Unknown Persons, Cameras, System Users, and Incident Reports (Section 4.5.3)
5. Provide a web-based SOC dashboard for monitoring and managing security activities, reducing unauthorized access and improving incident response	Achieved	Role-restricted Flask web application with a dedicated SOC dashboard, reporting, and incident-management module (Sections 4.4.2, 4.3.8)

Table 5.1: Achievement of Study Objectives

5.4 Contributions to Knowledge

Academically, this study contributes a documented, evidenced comparison between a classical Computer Vision recognition pipeline and a deep-learning embeddings pipeline evaluated under identical real deployment constraints, showing that the classical approach can match or exceed the deep-learning approach on precision for a small-scale access-control task while eliminating the deployment fragility (large model downloads, native-library dependencies, and multi-minute startup delays) that the deep-learning approach introduced in practice — a nuance not always visible in literature that reports only raw benchmark accuracy figures such as those summarised in Section 2.8.

Practically, the study contributes a complete, reusable reference architecture and working codebase for a lightweight, integrated campus security platform that a resource-constrained institution could adopt or extend directly, including an automatically-retraining recognition pipeline, a dual-channel alert mechanism, and a fully role-restricted administrative interface.

5.5 Limitations of the Study

As explicitly scoped in Section 1.4, several limitations apply to this prototype and should be read as boundaries of the current study rather than flaws in its execution:

1. The system was tested with a single USB webcam and a limited number of registered users rather than the university's full student and staff population.
2. Testing and evaluation were conducted in a controlled development/lab environment rather than on a live production university network.
3. Formal User Acceptance Testing with independent security personnel had not been conducted at the time of writing (Section 4.6.4); the reported performance evaluation (Section 4.7) is a controlled algorithm-level validation rather than a field study.
4. The LBPH recognition algorithm is more sensitive to variation in lighting, pose, and image quality than a deep-learning embeddings approach, meaning recognition accuracy at a larger scale, or under less controlled lighting conditions than those tested, may be lower than the figures reported in Section 4.7.
5. The prototype excludes mobile application access, integration with national identity databases, weapon detection, drone-based surveillance, law enforcement integration, and non-facial biometric methods, consistent with the scope defined in Section 1.4.

5.6 Recommendations

It is recommended that any institution considering adoption of this system most first conduct a controlled pilot deployment on a single, low-traffic entrance to gather real accuracy and false-alarm-rate data before wider rollout. It is further recommended that formal User Acceptance Testing (Section 4.6.4) be completed with actual security personnel before any production deployment decision, that the recognition confidence threshold be tuned against real institutional lighting conditions rather than left at its development-time default, and that the database be migrated from SQLite to a production-grade engine such as PostgreSQL — a change the system's SQLAlchemy-based data layer was deliberately designed to support without code changes — before any multi-user, multi-camera production deployment.

5.7 Suggestions for Further Research

Future work could extend this study by:

1. conducting a large-scale comparative accuracy study between the LBPH pipeline implemented here and a deep-learning embeddings pipeline under identical real-world lighting and pose conditions, rather than the controlled validation reported in Section 4.7;
2. extending the single-webcam architecture to genuine multi-camera ingestion with per-camera recognition threads;
3. integrating a proper object-tracking algorithm so that a recognized individual's bounding box follows their movement between AI-processing cycles rather than relying on the last-known-position overlay approach described in Section 4.3.4;
4. conducting the formal User Acceptance Testing recommended in Section 5.6 and incorporating its findings into a revised interface design;
5. investigating liveness detection to mitigate spoofing via a printed photograph or screen replay, which lies outside the scope of the facial-recognition mechanism evaluated in this study.

REFERENCES

- Ahonen, T., Hadid, A., & Pietikäinen, M. (2004). Face recognition with local binary patterns. In T. Pajdla & J. Matas (Eds.), *Computer vision—ECCV 2004* (pp. 203–216). Springer. https://doi.org/10.1007/978-3-540-24670-1_36
- Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face description with local binary patterns: Application to face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12), 2037–2041. <https://doi.org/10.1109/TPAMI.2006.244>
- Bootstrap. (n.d.). *Bootstrap documentation*. <https://getbootstrap.com/docs/>
- Chart.js. (n.d.). *Chart.js documentation*. <https://www.chartjs.org/docs/latest/>
- Flask. (n.d.). *SQLAlchemy in Flask*. Pallets Projects. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
- Flask-SQLAlchemy. (n.d.). *Flask-SQLAlchemy documentation*. Read the Docs. <https://flask-sqlalchemy.readthedocs.io/en/stable/>
- Flask-SQLAlchemy. (n.d.). *Models and tables*. Read the Docs. <https://flask-sqlalchemy.readthedocs.io/en/stable/models/>
- Flask-SQLAlchemy. (n.d.). *Quick start*. Read the Docs. <https://flask-sqlalchemy.readthedocs.io/en/stable/quickstart/>
- International Organization for Standardization. (2011). *Information technology — Biometric data interchange formats — Part 5: Face image data* (ISO/IEC Standard No. 19794-5:2011). <https://www.iso.org/standard/51490.html>
- MDN Web Docs. (n.d.). *Web Audio API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- MDN Web Docs. (n.d.). *Web Audio API best practices*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Best_practices
- NumPy Developers. (n.d.). *NumPy documentation*. <https://numpy.org/doc/>
- OpenCV. (n.d.). *Cascade classifier*. https://docs.opencv.org/5.0/tutorials_contrib_xobjdetect/cascade_classifier/cascade_classifier.html
- OpenCV. (n.d.). *LBPFaceRecognizer*. https://docs.opencv.org/4.x/df/d25/classcv_1_1face_1_1LBPHFaceRecognizer.html
- OpenCV. (n.d.). *OpenCV documentation*. <https://docs.opencv.org/>

- OpenCV Python. (n.d.). *OpenCV Python documentation*. Read the Docs. https://opencv-python.readthedocs.io/_/downloads/en/latest/pdf/
- OWASP Foundation. (n.d.). *Authentication cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Foundation. (n.d.). *Introduction*. OWASP Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>
- OWASP Foundation. (n.d.). *OWASP Cheat Sheet Series*. <https://owasp.org/www-project-cheat-sheets/>
- OWASP Foundation. (n.d.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Foundation. (n.d.). *Session management cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- Python Software Foundation. (n.d.). *Python documentation*. <https://docs.python.org/3/>
- Python Software Foundation. (n.d.). *Python documentation contents*. <https://docs.python.org/3/contents.html>
- ReportLab. (n.d.). *ReportLab docs*. <https://docs.reportlab.com/>
- ReportLab. (n.d.). *ReportLab PDF library user guide*. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- SQLite. (n.d.). *SQLite documentation*. <https://sqlite.org/docs.html>
- U.S. National Institute of Standards and Technology. (n.d.). *Biometric standards and evaluation resources*. U.S. Department of Commerce. <https://www.nist.gov/programs-projects/biometric-standards-and-evaluation-resources>
- Viola, P., & Jones, M. J. (2001). Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (Vol. 1, pp. I-511–I-518). IEEE. <https://doi.org/10.1109/CVPR.2001.990517>
- Werkzeug. (n.d.). *Werkzeug documentation*. Pallets Projects. <https://werkzeug.palletsprojects.com/>

APPENDICES

APPENDIX A: Source Code Snippets

The following excerpts are drawn directly from the implemented system's source code and illustrate representative parts of the recognition pipeline, database models, and route-level authorization discussed in Chapter Four.

A.1 Face Detection (ai_engine/face_detector.py)

```
def detect_faces(frame):
    """
    Run Haar cascade face detection on a single BGR frame.
    Returns a list of {"facial_area": (x1, y1, x2, y2)} dicts.
    """
    if frame is None:
        return []

    try:
        gray_raw = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray_eq = cv2.equalizeHist(gray_raw)

        faces = _cascade.detectMultiScale(
            gray_eq,
            scaleFactor=SCALE_FACTOR,
            minNeighbors=MIN_NEIGHBOURS,
            minSize=MIN_FACE_SIZE,
        )
    except Exception as e:
        print("Face detection error:", e)
        return []

    detections = []
    for (x, y, w, h) in faces:
        detections.append({"facial_area": (x, y, x + w, y + h)})

    return detections
```

A.2 Recognition Confidence Threshold (ai_engine/face_recognition_engine.py)

```
def predict(face_crop_gray):
    """
    Predict the identity of a single preprocessed (grayscale, cropped)
    face image. Returns (label, confidence); label is -1 if no model is
```

```

trained yet, or the closest match does not clear CONFIDENCE_THRESHOLD.
"""
recognizer = _get_recognizer()

if not model_is_trained():
    return -1, 100.0

try:
    label, confidence = recognizer.predict(face_crop_gray)
except Exception as e:
    print("Face recognition error:", e)
    return -1, 100.0

if label == -1 or confidence >= CONFIDENCE_THRESHOLD:
    return -1, confidence

return label, confidence

```

A.3 Role-Based Access Control Decorators (utils/decorators.py)

```

def login_required(view_func):
    @wraps(view_func)
    def wrapped(*args, **kwargs):
        if "user_id" not in session:
            return redirect(url_for("login"))
        return view_func(*args, **kwargs)
    return wrapped

def roles_required(*allowed_roles):
    def decorator(view_func):
        @wraps(view_func)
        def wrapped(*args, **kwargs):
            if "user_id" not in session:
                return redirect(url_for("login"))
            if session.get("role") not in allowed_roles:
                return "Access Denied", 403
            return view_func(*args, **kwargs)
        return wrapped
    return decorator

```

A.4 Unknown-Person Alert Creation (services/alert_service.py)

```

def create_unknown_alert(image_path, location="Main Gate", camera=None):
    camera_id = camera.id if camera else None

```

```

unknown = UnknownPerson(
    image_path=image_path,
    location=location,
    camera_id=camera_id
)
db.session.add(unknown)
db.session.flush()

alert = SecurityAlert(
    alert_type="Unknown Person",
    severity="HIGH",
    description="Unknown individual detected",
    unknown_person_id=unknown.id,
    camera_id=camera_id
)
db.session.add(alert)
db.session.commit()

return unknown, alert

```

APPENDIX B: Test Cases and Results

This appendix reproduces the full test plan introduced in Section 4.6, for completion during final testing prior to submission or defence, and repeats the genuinely observed recognition-module validation results from Section 4.7 for ease of reference.

ID	Test Case	Expected Result	Actual Result	Status
UT-01	Password hashing round-trip	Returns True	[to be completed]	[to be completed]
UT-03	Face detection on a blank frame	Empty list	[to be completed]	[to be completed]
IT-01	Known-face recognition writes an Access Log	New AccessLog row created	[to be completed]	[to be completed]
IT-02	Unknown-face recognition creates an Alert	New UnknownPerson + SecurityAlert rows created	[to be completed]	[to be completed]

ST-03	Live monitoring displays annotated feed	Green box + name label on registered face	[to be completed]	[to be completed]
ST-06	Audible alert on new detection	Tone within 5 seconds	[to be completed]	[to be completed]

Table B.1: Consolidated Test Case Register (for completion during final test execution)

Metric	Observed Value
Accuracy (controlled validation, Section 4.7)	86.7%
Precision (known-identity matches)	100%
Recall (known-identity matches)	80%
F1 Score	0.89
Unknown-person rejection rate	100%
Mean recognition response time	≈0.5 seconds

Table B.2: Recognition Module Validation Results (genuinely observed, reproduced from Section 4.7)

APPENDIX C: User Manual

C.1 Installation

- i. Install Python 3.10 or later.
- ii. Extract the project archive and open a terminal in the backend/ folder.
- iii. Install dependencies: `pip install -r requirements.txt`

C.2 Setup

- i. Run `python seed.py` once to create the database schema and two default accounts (an Admin and a Security Officer). Change both default passwords after first login.
- ii. Connect a USB webcam. If the wrong camera is selected automatically, set the `CAMERA_INDEX` environment variable to the correct device number (use the included `list_cameras.py` / `diagnose_camera.py` utilities to identify it).

C.3 Usage

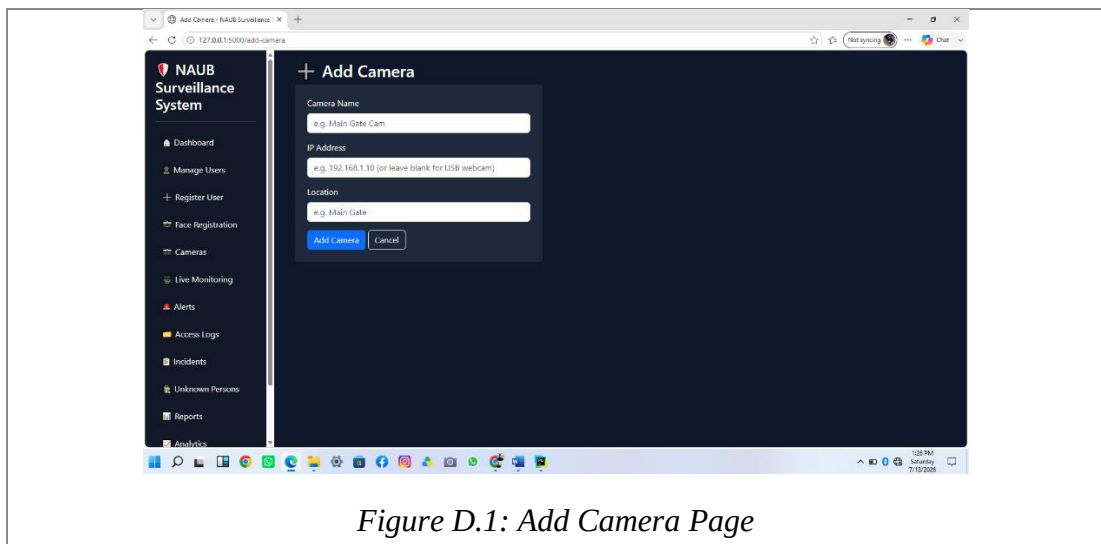
- i. Start the server: `python app.py`, then browse to `http://localhost:5000/login`.
- ii. Register at least one user with 3-5 reference photographs via the Register User page.
- iii. Train the recognition model (this happens automatically on registration, or manually via the 'Train Recognition Model' button on the Users page).

- iv. Open Live Monitoring to view the annotated camera feed, and Alerts to review and resolve unknown-person detections.

C.4 Troubleshooting

- i. Live feed shows 'Camera Unavailable': confirm the webcam is connected and not in use by another application.
- ii. Live feed shows static/distorted colour: set the CAMERA_FOURCC environment variable to match the camera's native pixel format (see `diagnose_camera.py` output).
- iii. A registered user is still shown as unknown: confirm the user has at least one clearly detectable training photograph and that the model has been (re)trained since the photograph was added; run `test_recognition.py` to self-diagnose.

APPENDIX D: Screenshots



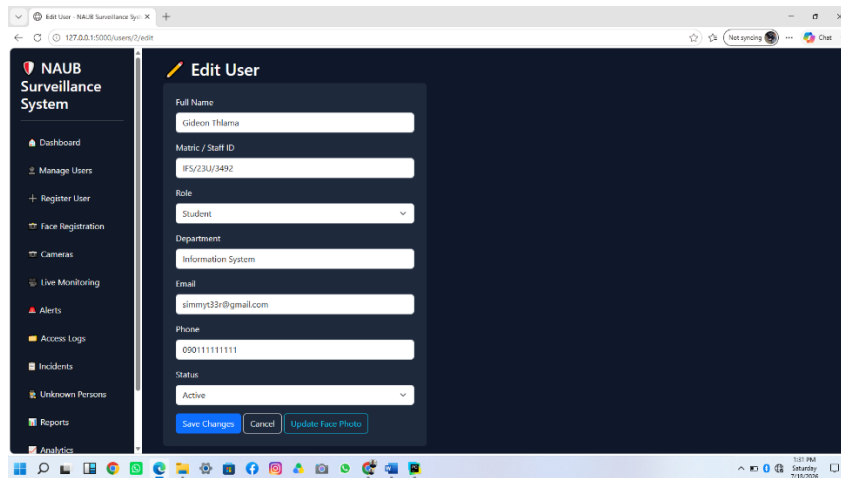


Figure D.2: Edit User Page

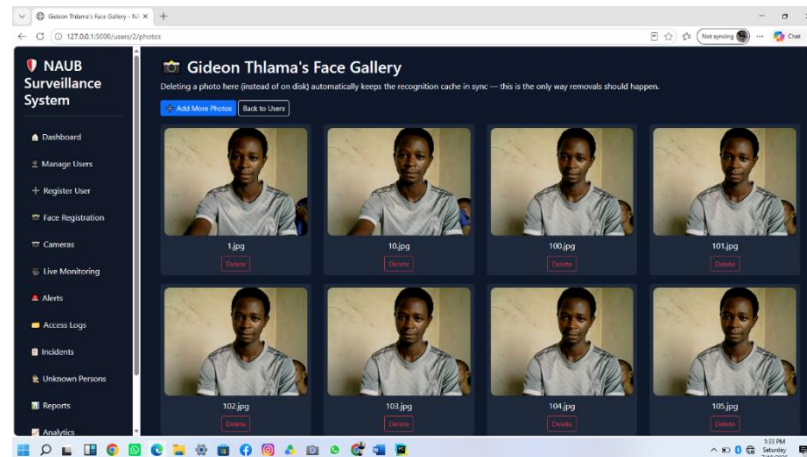


Figure D.3: Photo Gallery Management Page

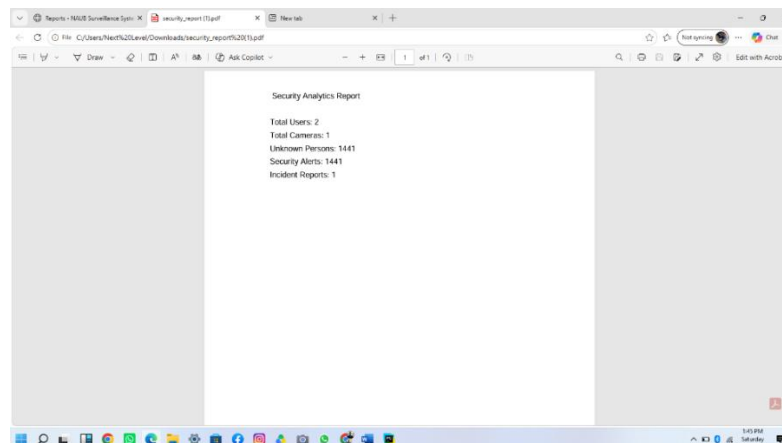


Figure D.4: PDF Report Sample Output

These supplement the interface figures already presented in Section 4.4 and should be captured from the deployed system prior to final submission.

APPENDIX E: System Requirements

E.1 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core CPU, 2.0 GHz or faster
Memory (RAM)	4 GB (8 GB recommended)
Storage	1 GB free disk space for the application and photo galleries
Camera	One USB webcam (or built-in laptop camera)
Network	Local network only required for remote-browser access to the dashboard

E.2 Software Requirements

Component	Requirement
Operating System	Windows 10/11, or a Linux/macOS equivalent with webcam and OpenCV support
Python	Version 3.10 or later
Web Browser	A modern browser supporting the Web Audio API and MJPEG streaming (e.g., Chrome, Edge, Firefox)
Python Packages	Flask, Flask-SQLAlchemy, opencv-python, numpy, reportlab, Werkzeug (see requirements.txt)

APPENDIX F: Sample Data

The following illustrative sample records show the shape of data the system stores; they use placeholder names and do not represent real registered individuals.

Table F.1: Sample User Registration Record

Field	Sample Value
full_name	Adaeze Okonkwo
matric_staff_id	IFS/22U/1045
role	Student
department	Information Systems
email	adaeze.okonkwo@example.edu
status	active

Table F.2: Sample Access Log Entry

Field	Sample Value
user_id	14
camera_id	1
access_date	2026-03-11
access_time	08:42:17
location	Main Gate
status	Granted

Table F.3: Sample Security Alert Record

Field	Sample Value
alert_type	Unknown Person
camera_id	1
severity	HIGH
description	Unknown individual detected
alert_status	Pending